

SOFTWARE DEVELOPMENT PROJECT

Final Report

CivicLink: Municipal Incident Reporting and Community Alert Platform

Prepared by:

Nthoroane KP

202328762

Department of Computer Science & Information Technology

Sol Plaatje University

Submitted to:

Dr Silas Verkijika

Lecturer: Project (NPRJ740)

Department of Computer Science & Information Technology

Sol Plaatje University

24-26 Scanlan St, New Park, Kimberley, 8301

30 October 2026

Table of Contents

EXECUTIVE SUMMARY

1 PROJECT CONTEXT

2 PROBLEM, PURPOSE AND SUCCESS CRITERIA

- 2.1 Problem Statement
- 2.2 Project Purpose
- 2.3 Success Criteria

3 SOLUTION OVERVIEW AND SCOPE

- 3.1 Solution Overview
- 3.2 Final Release Scope
- 3.3 Excluded and Deferred Capabilities
- 3.4 Known Limitations and Open Issues

4 SYSTEM REQUIREMENTS SPECIFICATION

- 4.1 Functional Requirements
- 4.2 Non-Functional Requirements
- 4.3 Business Rules
- 4.4 Acceptance Criteria for Included Requirements

5 SYSTEM DESIGN SPECIFICATION

- 5.1 Use Cases and Workflows
- 5.2 System Architecture and Technology Stack
- 5.3 Key Architecture and Design Decisions
- 5.4 Application Class Design
- 5.5 User Interface and Interaction Design
- 5.6 Data Design
- 5.7 Security, Privacy and Access Control
- 5.8 Deployment and Runtime Design

6 SYSTEM IMPLEMENTATION

- 6.1 Codebase and Repository Structure
- 6.2 Implementation of Core Capabilities

6.3 APIs, Integrations and External Dependencies

6.4 AI / Machine-Learning Component

7 VERIFICATION AND TESTING

7.1 Test Approach

7.2 Functional and Business-Rule Verification

7.3 Non-Functional and Security Verification

7.4 Defects and Known Issues

8 SYSTEM SETUP AND OPERATION

9 SOLUTION EVALUATION

10 REFERENCES

11 APPENDICES

11.1 GitHub Repository

11.2 Deployed System

11.3 Additional Supporting Material

List of Figures

Figure 1: CivicLink system context diagram

Figure 2: Use-case diagram of the final release

Figure 3: Activity diagram, W1 submit incident report

Figure 4: Sequence diagram, W1 submit incident report

Figure 5: Activity diagram, W2 triage, assign, complete and confirm

Figure 6: Sequence diagram, W2 assign and complete

Figure 7: State diagram of the Report entity

Figure 8: State diagram of the Task entity

Figure 9: Logical architecture

Figure 10a: Class diagram (part 1), domain entities and relationships

Figure 10b: Class diagram (part 2), application services and domain logic

Figure 11: Entry: public home, sign in, guest account

Figure 12: Resident reporting (W1): details, validation errors, duplicate

Figure 13: Official dashboard

Figure 14: Official, triage and assignment

Figure 15: Official, bulk assignment

Figure 16: Empty state in the report queue

Figure 17: Resolution (W2): worker completion, resolved report, rating

Figure 18: Official, alert composer with preview

Figure 19: Official, scheduled alerts

Figure 20: Resident community features: alerts, "Me too", offline report

Figure 21: Service-delivery analytics

Figure 22: Public transparency page

Figure 23: Administrator, user management

Figure 24: Administrator, SMS outbox

Figure 25: Access and accounts: permission denied, Google first sign-in, POPIA deletion

Figure 26: Firestore document model

Figure 27: Deployment diagram

List of Tables

Table 1: Success criteria

Table 2: Final release scope

Table 3: Excluded capabilities

Table 4: Known limitations

Table 5: Functional requirements

Table 6: Non-functional requirements

Table 7: Business rules

Table 8: Component responsibilities and technology stack

Table 9: Supporting decisions

Table 10: User interface evidence

Table 11: Data dictionary (principal collections)

Table 12: Role / permission matrix (✓ allowed; scope in brackets)

Table 13: Security controls

Table 14: Repository structure

Table 15: Capability implementation

Table 16: Callable command API

Table 17: Test levels

Table 18: Functional and business-rule test results

Table 19: Non-functional verification results

Table 20: Significant defects found and resolved

Table 21: Demo accounts

Table 22: Evaluation against the success criteria

Table 23: Abbreviations

Table 24: Requirements traceability (condensed from docs/trackers/traceability-matrix.md)

Table 25: Complete automated test results

EXECUTIVE SUMMARY

Sol Plaatje Local Municipality in Kimberley gets reports of burst pipes, power outages, potholes and illegal dumping by phone, at walk-in counters and through community WhatsApp groups. These reports are written down by hand, usually without an exact location, and the same problem is often reported many times. Field teams are sent out by phone with no record of what happened next, and residents hear nothing unless they call again. When municipal workers went on strike on 31 March 2026 and blocked roads across the city, the municipality had no official way to warn people.

I built CivicLink to put this whole process into one web application. **Residents** report a problem from their phone with a map pin, photos and a severity, get a reference number and can follow the report until it is fixed. **Officials** sort the reports for their department, assign them to field workers, escalate work that is running late and send alerts to specific suburbs. **Field workers** get their tasks on their phones and close each one with notes and a photo of the repair. **Administrators** manage staff accounts, departments and categories. Members of the public can also use it without an account.

The final release covers all 39 functional requirements, including all 14 Must-haves. The main lifecycle works from start to finish: a report is submitted, checked for duplicates, triaged, assigned, completed in the field and then confirmed (with a rating) or reopened by the resident. Around that, the system has real-time and push notifications, suburb alerts that can be sent now or scheduled (with SMS for emergencies), a community map with a "Me too" button, live analytics, a public transparency page, automatic SLA escalation, bulk assignment and a full audit trail. It can be installed on a phone, saves reports offline and sends them once the phone reconnects, supports Google sign-in, and lets residents delete their account in line with POPIA. Anyone can open the site without signing in to see alerts and open issues near them, and can report a problem as a guest.

The front end is React and TypeScript; the back end is Firebase Authentication, Cloud Firestore and a Node.js command API. Every change to data happens on the server inside a database transaction, and Security Rules control who can read what. Everything runs on free tiers (Vercel and Firebase's Spark plan), and the same system can be run locally on the Firebase Emulator Suite with two commands and no cloud account. I tested it with 185 automated tests (unit, security-rule, API, end-to-end and non-functional) that run on every push, and all five success criteria were met.

Live system: https://civiclink-five.vercel.app

Source code: https://github.com/altechemist/civiclink (public, release v1.0.0)

1. PROJECT CONTEXT

Application domain. Municipal service delivery: the reporting, dispatch and resolution of infrastructure and community incidents (water, electricity, roads, waste and public safety), and communication between a municipality and its residents.

Primary users and client. The intended client is Sol Plaatje Local Municipality, Northern Cape. Its users are residents of the municipal area (Kimberley, Galeshewe, Roodepan, Ritchie, Platfontein and surrounding areas), municipal officials in the five service departments, field workers who carry out repairs, and system administrators.

Basis of the project. The project uses a **real organisation and real events**: the municipality, its departments and the 31 March 2026 strike. The requirements came from a **simulated** interview with a senior infrastructure official, an analysis of how the municipality handles reports day to day, and published sources. The municipality did not commission CivicLink and is not using it.

2. PROBLEM, PURPOSE AND SUCCESS CRITERIA

2.1 Problem Statement

At the moment residents have no reliable digital way to report service failures, and the municipality has no single system to manage them. In Phase 1 I found these problems:

- **No precise capture.** Calls are written on paper or in spreadsheets, without GPS coordinates or photos, so field teams are sent to the wrong place and repeat visits are common, especially for illegal dumping.
- **Duplicate reports.** The same burst pipe or outage is reported many times across channels, which wastes call-centre time and distorts priorities.
- **No accountability for dispatch.** Supervisors phone field workers directly, leaving no record of who was assigned, when work started, or whether it was finished.
- **No feedback to residents.** Residents are not told when a report is accepted or resolved, so they call back repeatedly. This adds to the call-centre load and erodes trust.
- **No targeted public communication.** During the 31 March 2026 strike, road closures were communicated only by radio and informal WhatsApp groups. Residents and taxi commuters received no official guidance.

- **No management visibility.** Without data, the municipality cannot measure response times, compare departments or identify hotspots.

In the stakeholder questionnaire, burst pipes took 24–72 hours to fix, mostly because reports went to the wrong team. None of these problems comes from one person doing their job badly; they come from a manual, scattered process. A shared system that records every report, assignment and status change in one place, applies the routing rules and tells the affected people what is happening deals with each of them directly.

Published sources back this up. Sol Plaatje has 270,078 residents (Statistics South Africa, 2023). The municipality's own plan sets a turnaround strategy for water and electricity losses (Sol Plaatje Local Municipality, 2022), and the municipal manager has reported non-revenue physical and technical water losses of 60%, with the bulk pipelines from Riverton leaking because of corrosion and sabotage (Noordkaap Bulletin, 2024). In 2026 a resident's published opinion piece described burst pipes as "a near-daily occurrence", with a reported burst still running hours later (Diamond Fields Advertiser, 2026), the same response-time gap identified in the interview. Nationally, only 34 of South Africa's municipalities (13%) obtained clean audits in 2022-23, and the Auditor-General reports weak controls over performance information and service delivery (Auditor-General South Africa, 2024). So the need for accountable, traceable records of service delivery is not just a local one.

2.2 Project Purpose

The aim of CivicLink is to replace the manual, scattered reporting process with one accountable workflow. A resident should be able to report a problem with its location and a photo in under a minute and follow it until it is fixed. Officials should be able to send the work to the right team and warn the affected suburbs straight away. Field workers should be able to close a task with proof, and every step should leave a record that can be checked later.

2.3 Success Criteria

Table 1: Success criteria

ID	Success criterion	How it is verified
SC-01	The complete incident lifecycle runs end-to-end on the submitted system: a resident submits, an official triages and assigns, a field worker completes with proof, and the resident sees the report resolved with a full status history.	End-to-end test E2E-01; demonstration
SC-02	The key business rules are enforced on the server, not only in the interface: duplicate detection, allowed status transitions, one active task per report, valid assignment and SLA escalation.	Unit and integration tests for BR-03 to BR-11

ID	Success criterion	How it is verified
SC-03	Role-based access control holds: each role can perform exactly the functions in the permission matrix and nothing else.	Security-rule tests RT-*, API authorisation tests, E2E-04
SC-04	Residents receive timely, targeted information: a status change or a suburb alert appears in the resident's app within 5 seconds, without refreshing.	NFT-05, E2E-03
SC-05	Officials have operational visibility from live data: dashboard metrics match the known dataset.	UT-ANL-01, IT-ANL-01

3. SOLUTION OVERVIEW AND SCOPE

3.1 Solution Overview

CivicLink is a responsive web application. Inside the system boundary are the React app (with separate areas for residents, officials, field workers and administrators), a Node.js command API (Cloud Functions locally, Vercel serverless functions when deployed), Cloud Firestore and Firebase Authentication. Outside it are the four user roles and members of the public, who can view alerts, open issues and figures and report as guests without an account. The external services are OpenStreetMap for map tiles, Google sign-in, Firebase Cloud Messaging for push, Twilio for SMS (optional), the Vercel CDN, the GitHub Actions schedule and the phone's location service.

The main functional areas are:

- **Incident reporting:** a three-step report form with category, map location, suburb, landmark, description, severity and up to three photos; reference numbers; and duplicate detection within 100 m.
- **Tracking and feedback:** a live status lifecycle, an audit timeline, in-app notifications, confirmation or reopening by the resident, and automatic closure.
- **Dispatch:** a department-scoped report queue with triage, rejection, assignment, reassignment and escalation, plus internal staff notes.
- **Field work:** a mobile task list where the worker accepts, starts and completes tasks with notes and proof photos.
- **Community communication:** suburb-targeted alerts, sent now or scheduled, with delivery and read receipts, push and SMS for emergencies; a community incident map with approximate locations and "Me too".

- **Management and administration:** live analytics on volumes, resolution time and SLA compliance; bulk assignment; user, role, department and category management; and a system log with the SMS outbox.
- **Public access:** a public home page with the alerts and open issues around the visitor, reporting without an account (as a guest), and a transparency page, with anonymous figures on open issues, time to fix, SLA compliance and resident satisfaction.

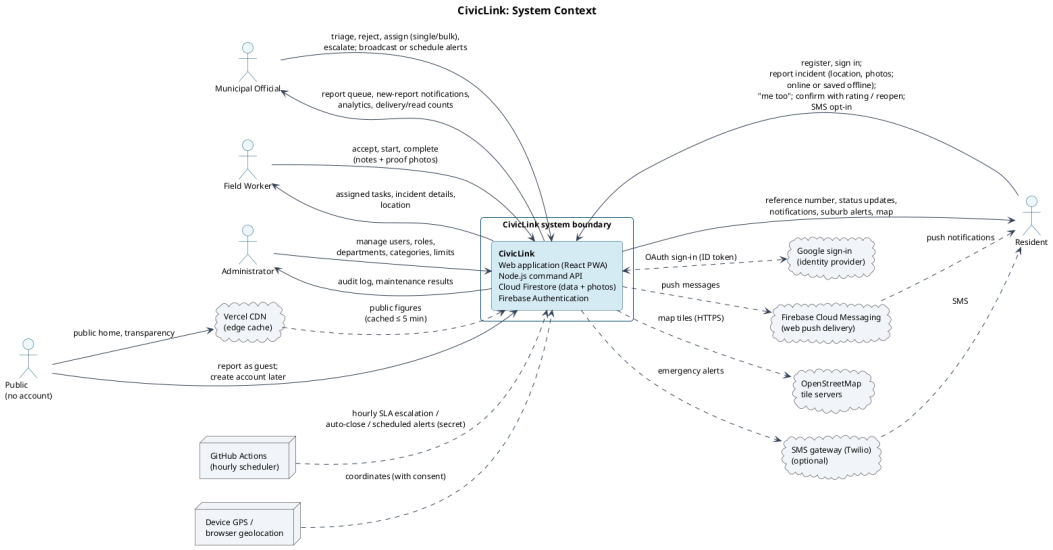


Figure 1: CivicLink system context diagram

3.2 Final Release Scope

The submitted release (tag v1.0.0) includes the capabilities below. Each one links to its requirements in Section 4 and its tests in Section 7.

Table 2: Final release scope

Capability	Requirements	Priority
Resident registration, sign-in and role-based routing; account deactivation	FR-01, FR-02, FR-20	Must
Located incident report with photos; reference number; duplicate detection	FR-03, FR-04, FR-05	Must
My reports with live status lifecycle and audit timeline	FR-06	Must
Department report queue with filters, search and CSV export	FR-08, FR-26	Must / Could
Triage, reject, assign field worker	FR-09, FR-10	Must
Field-worker task list; accept, start, complete with proof	FR-11	Must
Real-time in-app notifications	FR-15	Must
Suburb-targeted alert broadcast and resident alerts feed	FR-16, FR-17	Must

Capability	Requirements	Priority
Reassignment; manual and SLA-based escalation; resident confirm/reopen; auto-close	FR-12, FR-13, FR-14	Should
Community incident map (no personal details)	FR-07	Should
Alert history with delivery and read counts	FR-18	Should
Service-delivery analytics	FR-19	Should
Department and category configuration	FR-21	Should
Internal staff comments; system log	FR-22, FR-23	Could
Google sign-in; password reset; profile editing; POPIA account deletion	FR-29–FR-32	Should
"Me too" confirmation of open issues; installable app with offline shell	FR-33, FR-34	Should
Web push notifications; SMS for emergency alerts	FR-24, FR-25	Could
Offline report drafts sent on reconnect	FR-27	Could
Scheduled alerts; bulk assignment	FR-28, FR-35	Could
Public transparency page; resident service rating	FR-36, FR-37	Could
Public home page (alerts, open-issue map, figures); reporting as a guest with optional account	FR-38, FR-39	Should

3.3 Excluded and Deferred Capabilities

Every functional requirement made it into the release. At the Phase 3 baseline I had deferred four capabilities (SMS, offline drafts, scheduled alerts and bulk assignment), but once the core was finished and tested I had time to add them back (CR-33 to CR-36). Only the following is still outside the release:

Table 3: Excluded capabilities

Capability	Status	Reason
Real SMS sending in the demo deployment	Simulated	Every SMS costs money. Without gateway credentials each SMS is recorded as Simulated; three environment variables switch to Twilio (CR-34)
Native Android/iOS apps	Excluded	The responsive web app works on low-end phones without app-store distribution
Google Maps and reverse geocoding	Excluded (replaced)	Replaced by Leaflet and OpenStreetMap (ADR-0004)
99.5% availability, 10 000 concurrent users, multi-region failover	Excluded	Production targets from Phases 1–2 that cannot be verified in a student environment
AI/ML prediction	Excluded	Not part of the product

3.4 Known Limitations and Open Issues

Everything in Section 3.2 is complete and tested. These are the limitations that still affect included features:

Table 4: Known limitations

Area	Limitation	Impact	Status / workaround
Push (FR-24)	Push is best-effort and per device; on iPhone it works only for the installed app (iOS 16.4+)	A resident who has not turned push on sees updates only in the app	In-app notifications remain the guaranteed channel; emergency alerts can also go by SMS
SMS (FR-25)	No paid gateway in the demo (the Twilio trial sends a predefined template text instead of the alert; the real text is kept in the outbox); SMS are sent inside the alert request	Messages are recorded as Simulated; a very large emergency broadcast would make the official wait	Twilio by configuration; a queue at production scale
Offline (FR-27, FR-34)	Offline, only new reports can be saved; other actions need a connection, and the map shows only areas viewed before	A saved report gets its reference number when it is sent	Sent automatically on reconnect, stored once
Scheduled rules (FR-13, FR-14, FR-28)	The emulator does not fire schedules; the free deployment's schedule runs hourly	Locally an administrator selects "Run scheduled rules now"; a scheduled alert can be up to an hour late when nobody is using the site	While the site is in use, due alerts are published within about a minute (background check after API requests)
Alert broadcast (FR-16)	Delivery records are written in batches after the alert document, not in one transaction	A failure during a broadcast could leave the recipient count higher than the stored delivery records	Acceptable at municipal scale; records can be re-derived from the suburb query
Guest accounts (FR-39)	Guest sessions that never report are not deleted automatically	Unused anonymous accounts accumulate in Firebase Authentication (no personal data is stored for them)	A scheduled clean-up of empty guest accounts is a production follow-up
Location (FR-03)	No street address; location is a map pin plus suburb and optional landmark	Officials rely on the pin and the landmark text	Deliberate (ADR-0004)

4. SYSTEM REQUIREMENTS SPECIFICATION

The requirements below are the consolidated baseline that controls the project. Changes from the Phase 1–3 documents (for example the added Administrator actor, the eight-state report lifecycle and the rewritten non-functional requirements) are recorded as change requests CR-01 to CR-45 in docs/baseline/change-log.md.

4.1 Functional Requirements

Table 5: Functional requirements

ID	Functional requirement	Priority	Release status
FR-01	The system shall allow a member of the public to register as a Resident with name, email, password, suburb and optional phone number.	Must Have	Included
FR-02	The system shall authenticate users by email and password and route each user to the interface for their role; deactivated accounts are refused.	Must Have	Included
FR-03	The system shall allow an authenticated Resident to submit an incident report with category, description, map location, suburb, optional landmark, severity and 0–3 photos.	Must Have	Included
FR-04	The system shall give each accepted report a unique reference (CL-YYYY-NNNN), status Submitted, a default department derived from its category, and a first history entry.	Must Have	Included
FR-05	The system shall detect when a new report duplicates an open report, link it to the original and tell the resident the original's reference.	Must Have	Included
FR-06	The system shall allow a Resident to list their reports (All / Active / Resolved) and view a report's details and full status timeline.	Must Have	Included
FR-07	The system shall show a community map of open reports with category filtering, without revealing reporter identity or photos.	Should Have	Included
FR-08	The system shall allow an Official to view the report queue with filters (status, category, department, severity, unassigned) and search by reference.	Must Have	Included
FR-09	The system shall allow an Official to triage a Submitted report (to Under Review, confirming or changing the department) or reject it with a reason.	Must Have	Included
FR-10	The system shall allow an Official to assign a report under review or escalated to an active field worker in its department, creating a task and moving the report to In Progress.	Must Have	Included
FR-11	The system shall allow a Field Worker to view, accept, start and	Must Have	Included

ID	Functional requirement	Priority	Release status
	complete their tasks; completion requires notes and at least one proof photo and resolves the report.		
FR-12	The system shall allow an Official to reassign an active task to another eligible field worker.	Should Have	Included
FR-13	The system shall escalate open reports that exceed their department's SLA hours automatically, and allow an Official to escalate manually with a reason.	Should Have	Included
FR-14	The system shall allow the reporting Resident to confirm or reopen a resolved report within 7 days, and shall close resolved reports automatically after 7 days.	Should Have	Included
FR-15	The system shall notify users in-app in real time: residents on every status change of their (or a followed) report, officials on new reports, and field workers on assignment.	Must Have	Included
FR-16	The system shall allow an Official to broadcast an alert with type, title, message, target suburbs (or all) and expiry, recording a delivery record for each targeted resident.	Must Have	Included
FR-17	The system shall show Residents the active, unexpired alerts for their suburb, emergencies first, with unread markers and read receipts.	Must Have	Included
FR-18	The system shall allow an Official to view sent alerts with recipient and read counts and to end an alert early.	Should Have	Included
FR-19	The system shall show Officials an analytics dashboard computed from live data: open reports, resolution time, SLA compliance per department and the daily trend.	Should Have	Included
FR-20	The system shall allow an Administrator to create staff accounts, change roles and deactivate or reactivate accounts.	Must Have	Included
FR-21	The system shall allow an Administrator to manage departments (name, SLA hours, active) and categories (name, default department, active).	Should Have	Included
FR-22	The system shall allow Officials and Field Workers to add internal comments to a report that residents cannot see.	Could Have	Included
FR-23	The system shall record significant system events and show them to Administrators.	Could Have	Included
FR-24	The system shall send web push notifications for status changes, task assignments and suburb alerts to devices where the user has turned them on.	Could Have	Included
FR-25	The system shall also send emergency alerts by SMS to residents who opted in with a mobile number, and record each message's	Could Have	Included

ID	Functional requirement	Priority	Release status
	delivery status.		
FR-26	The system shall allow an Official to export the filtered report queue as CSV.	Could Have	Included
FR-27	The system shall let residents save reports without a connection and send them automatically, exactly once, when the connection returns.	Could Have	Included
FR-28	The system shall allow an Official to schedule an alert for a future time, see scheduled alerts and cancel them before publication.	Could Have	Included
FR-29	The system shall allow users to sign in with a Google account; on first sign-in a new user completes a resident profile.	Should Have	Included
FR-30	The system shall allow users to request a password-reset email without revealing whether an account exists.	Should Have	Included
FR-31	The system shall allow users to update their name and phone number, and residents their suburb.	Should Have	Included
FR-32	The system shall allow a Resident to delete their account; personal data is erased and their reports are kept anonymised.	Should Have	Included
FR-33	The system shall allow a Resident to confirm an existing open issue from the community map ("Me too"), raising its affected count and following its progress.	Should Have	Included
FR-34	The system shall be installable on a phone's home screen and shall open, with previously viewed information, without a connection.	Should Have	Included
FR-35	The system shall allow an Official to assign several triaged reports of their department to one field worker in a single step, reporting any that could not be assigned.	Could Have	Included
FR-36	The system shall publish anonymous performance figures (open issues, time to fix, SLA compliance per department, satisfaction) on a public page that needs no sign-in.	Could Have	Included
FR-37	The system shall allow a Resident confirming a fix to rate the service from 1 to 5 stars with an optional comment.	Could Have	Included
FR-38	The system shall show visitors who are not signed in a public home page with the active alerts, a map of open issues (approximate locations) filterable by area, and headline figures.	Should Have	Included
FR-39	The system shall allow anyone to report an incident without an account (as a guest), follow it on the same device, and later keep it by creating an account or signing in to an existing one.	Should Have	Included

4.2 Non-Functional Requirements

Non-functional requirements NFR-01 to NFR-12 are verified in the assessment environment: the local Firebase Emulator Suite with the seeded dataset (150 reports, 15 users) on the latest Chrome and Firefox. NFR-13 and NFR-14 are properties of the hosting configuration and are verified against the deployed site.

Table 6: Non-functional requirements

ID	Quality attribute	Requirement	Verification
NFR-01	Security: authorisation	Every role-restricted read or write is enforced server-side by Security Rules and Cloud Function role checks, independently of the user interface.	Security-rule tests, authorisation tests
NFR-02	Security: authentication	Passwords are handled only by Firebase Authentication (minimum 8 characters); no credential is stored in Firestore or the repository; roles are carried as signed custom claims.	Code review, rule tests, secret scan
NFR-03	Security: input validation	All commands validate input on the server against shared schemas; invalid input is rejected with field-level messages and no data change.	Integration tests
NFR-04	Security: secrets	No secrets are kept in source control; configuration comes from environment variables with a committed .env.example.	Secret scan (gitleaks)
NFR-05	Performance: propagation	A new report appears on the official's queue, and a status change appears on the resident's screen, within 5 seconds without a refresh.	Timing test NFT-05
NFR-06	Performance: responsiveness	Key screens become interactive within 3 seconds; commands complete within 2 seconds (95th percentile of 50 calls).	Timing test NFT-06
NFR-07	Usability: mobile	Resident and field-worker screens work at 360×640 with no horizontal scrolling; submitting a report takes at most 3 steps.	NFT-07
NFR-08	Accessibility	Key pages have no serious or critical WCAG 2.1 A/AA violations in an automated axe-core audit.	NFT-08
NFR-09	Compatibility	Core flows pass on the latest Chrome and Firefox.	End-to-end tests on both browsers
NFR-10	Integrity	Multi-document operations are atomic; conflicting concurrent commands result in	Concurrency tests

ID	Quality attribute	Requirement	Verification
		exactly one success; failures leave no partial data.	
NFR-11	Privacy (POPIA)	Residents' identity, contact details and photos are visible only to the reporter and staff; public map data contains no personal information.	Rule tests, data classification
NFR-12	Maintainability	TypeScript strict mode, lint-clean, shared domain logic with at least 80% line coverage, and CI on every push.	CI, coverage report
NFR-13	Performance: caching and CDN	Hashed assets are cached for a year as immutable; the app shell is revalidated on every visit; the public figures are served from the CDN edge cache (at most 5 minutes old); map tiles already viewed are cached on the device.	NFT-11 on the deployed site
NFR-14	Security: browser hardening	The deployed site sends a Content Security Policy without `unsafe-eval` or inline scripts, HSTS, frame denial and nosniff, and the app runs without CSP violations.	NFT-12 on the deployed site

4.3 Business Rules

Table 7: Business rules

ID	Rule
BR-01	Only an authenticated, active user with role Resident may submit a report. A guest (FR-39) is an anonymous sign-in given a guest Resident profile by the server, so it is authenticated for this rule.
BR-02	A report needs an active category, a description of 10–300 characters, a location inside the Sol Plaatje municipal bounds, a suburb from the configured list and a severity. It may carry 0–3 photos, each a real JPEG/PNG/WebP image (checked by its content) of at most 700 KB once compressed; residents may pick files of up to 15 MB, which the app compresses.
BR-03	A new report is a duplicate if an open report with the same category exists within 100 m and was created in the last 72 hours. The duplicate is stored as Duplicate, linked to the original; the original's affected count is incremented and the resident follows it.
BR-04	References are CL-<year>-<sequence>, allocated in a transaction, unique and never reused.
BR-05	A report's status may change only along the transitions of the report state machine (Figure 7). Every change appends an immutable history entry recording the actor, role, note and time.
BR-06	A report has at most one active task at any time.
BR-07	A report may be assigned only to an active Field Worker of the report's department, and only while it is Under Review, Escalated or (for reassignment) In Progress.
BR-08	Only the assigned Field Worker may act on a task, and only in the order Assigned → Accepted → In

ID	Rule
	Progress → Completed.
BR-09	Completion requires notes of at least 10 characters and 1–3 proof photos, and sets the report to Resolved in the same transaction.
BR-10	An open report not resolved within its department's SLA hours is escalated automatically (once); officials may escalate manually with a reason.
BR-11	Only the reporting Resident may confirm or reopen a resolved report, within 7 days of resolution; afterwards it closes automatically. Closed, Rejected and Duplicate are terminal.
BR-12	Rejection requires a reason of 10–300 characters and is allowed only for Submitted or Under Review reports.
BR-13	Only Officials and Administrators may create alerts; an alert needs a title (≤ 100), message (≤ 500), at least one target suburb or All, and an expiry in the future and within 30 days.
BR-14	A resident sees an alert only while it is active and unexpired and targets their suburb or All.
BR-15	Only Administrators manage accounts and configuration; an Administrator cannot demote or deactivate their own account.
BR-16	Self-registration always creates a Resident; staff roles are granted only by an Administrator; a deactivated account cannot sign in or call any function.
BR-17	Reporter identity, contact details and photos are visible only to the reporter and staff; other residents see category, status, approximate location (about 100 m) and age only.
BR-18	Internal comments are visible only to staff.
BR-19	Inactive departments and categories cannot be selected for new reports, triage or assignment.
BR-20	Officials may act only on reports of their own department; Administrators may act on any.
BR-21	A resident may submit at most N reports in any rolling hour; N is set by an Administrator (1–100, default 10).
BR-23	A resident may confirm ("Me too") an issue once, only while it is open, and not their own report.
BR-24	A scheduled alert starts 10 minutes to 30 days ahead and before its expiry; it is published once and keeps its identifier; it can be cancelled only while still scheduled; one that expires before publication is cancelled.
BR-26	A guest may submit at most 3 reports per rolling hour; at most 10 guest sessions may start per network address per hour. Guests cannot opt in to SMS or edit a profile. They keep their reports by creating an account in the same session, or by moving them into an existing account with a one-time code valid for 15 minutes.
BR-25	Only Emergency alerts go by SMS, only to active residents of the target suburbs who opted in and gave a mobile number, one 160-character message each. SMS records are visible to Administrators only and are erased with the account.
BR-22	When a resident deletes their account, their name, email, phone, photos, notifications, alert receipts, profile and sign-in are erased; their reports are kept for service records as "Former resident". Staff accounts are deactivated, never self-deleted.

4.4 Acceptance Criteria for Included Requirements

FR-01 Register. Valid details create a Resident account and profile. A duplicate email is refused with "Email already registered". A password shorter than 8 characters or a missing suburb is refused with a field message. A role sent by the client is ignored.

FR-02 Sign in. Valid credentials open the area for the user's role. Wrong credentials show a generic error. A deactivated account cannot sign in or call any function. Opening another role's page shows the access-denied page.

FR-03 Submit report. Every BR-02 rule is checked in the form and again on the server, with field-level messages. A location outside the municipality, a fourth photo, a file over 15 MB or a file that is not really an image is refused. On success the resident sees the reference number.

FR-04 Reference. Concurrent submissions receive distinct, sequential references. A new report is Submitted, routed to its category's department, and has one history entry.

FR-05 Duplicate. A report of the same category within 100 m of an open report from the last 72 hours is linked as a duplicate, and the resident is shown the original's reference. Beyond 100 m, a different category, or a closed original produces a new report.

FR-06 My reports. Only the resident's own (or followed) reports are shown. The tabs filter correctly. The timeline is chronological with actor role and note. Another resident's report cannot be opened. An empty state is shown when there are no reports.

FR-07 Map. Open reports appear as category-coloured markers with a working filter. Popups show category, status, suburb and age only. The public data contains no reporter, description or photos.

FR-08 Queue. Officials see only their department's reports (Administrators see all). Filters combine. Searching by reference finds the report. Unassigned reports are flagged. An empty result shows an empty state.

FR-09 Triage / reject. Triage moves Submitted to Under Review and records history. Rejection requires a reason. Triage from any other status, or by an official of another department, is refused.

FR-10 Assign. Only active workers of the report's department can be chosen. Assignment creates an Assigned task, moves the report to In Progress, and notifies the worker and resident. A report that already has an active task cannot be assigned again. Of two simultaneous assignments, exactly one succeeds.

FR-11 Field worker. Workers see only their own tasks. Steps must happen in order. Completion without notes or a proof photo is refused. Completion resolves the report and notifies the resident. Another worker cannot act on the task.

FR-12 Reassign. The old task becomes Reassigned and a new task is created. Reassigning to the same worker, or when there is no active task, is refused. A reassigned task can no longer be completed.

FR-13 Escalate. A report past its SLA is escalated by the next scheduled run, exactly once. Manual escalation requires a reason. An escalated report can still be assigned.

FR-14 Confirm / reopen. Within 7 days the reporter can confirm (Closed) or reopen with a reason (Under Review, officials notified). After 7 days both are refused and the schedule closes the report. Other residents cannot act on it.

FR-15 Notifications. Each lifecycle event creates notifications for the correct recipients. The unread badge updates live. Opening a notification marks it read and navigates to the record.

FR-16 Broadcast. BR-13 is enforced with field messages. A valid alert creates one delivery record per active resident in the target suburbs. Residents and field workers cannot broadcast.

FR-17 Alerts feed. Residents see only active, unexpired alerts for their suburb or All, emergencies first. Reading records a receipt once per resident.

FR-18 Alert history. Recipient and read counts match the delivery records. Ending an alert hides it from residents immediately.

FR-19 Analytics. The metrics match the dataset. Empty data shows "no data" rather than an invalid number. Residents cannot access analytics.

FR-20 User administration. Staff accounts are created with the correct role and department. Role changes and deactivation take effect immediately. Administrators cannot demote or deactivate themselves. Non-administrators are refused.

FR-21 Configuration. SLA hours must be a whole number from 1 to 720. Names are unique. A deactivated category disappears from the report form. The hourly report limit (BR-21) must be from 1 to 100.

FR-22 Internal comments. Officials of the report's department, its assigned field worker and Admins can add a note (1–1000 characters), and notes appear in time order with author and role. Residents and staff of other departments cannot read or add notes.

FR-23 System log. Administrative actions (account, role, configuration and settings changes), scheduled runs and unexpected server errors are recorded with level, source, time and actor. Only Administrators can view the log. Entries are never edited.

FR-26 CSV export. The export contains exactly the reports matching the current queue filters and search, with reference, category, department, suburb, severity, status, assignee, affected count and submitted/resolved times. Values with commas, quotes or line breaks are quoted correctly. The button is disabled when no reports match.

FR-24 Web push. With push turned on, a committed status change, new task or suburb alert produces one notification per device, and clicking it opens the related screen. A failed command sends nothing. Turning push off, or deleting the account, removes the device. The option is hidden where push is unavailable.

FR-33 Me too. From the map popup of an open issue, a resident confirms it once: the affected count increases, they follow its progress, and the timeline notes it. A repeat reports "already confirmed". Their own report, closed issues and staff callers are refused.

FR-34 Installable and offline. The browser offers to install CivicLink. With no connection, the app opens, previously viewed information still shows, and an offline notice explains that new reports are kept on the phone.

FR-25 SMS for emergency alerts. Opting in needs a valid SA mobile number, and removing the number ends the opt-in. An Emergency alert creates one SMS record per opted-in resident of the target suburbs (international format, at most 160 characters); other types create none. Each record is Sent, Simulated or Failed; a gateway failure does not stop the alert. Only Administrators can read SMS records.

FR-27 Offline drafts. With no connection, the last step offers "Save and send later"; the report is listed on the home screen with a discard option and is sent automatically on reconnect, showing its reference number. Sending it twice stores it once. A refused report stays listed with the reason. Another account on the same phone never sends it.

FR-28 Scheduled alerts. A publish time 10 minutes to 30 days ahead and before expiry is accepted. Until then residents cannot see the alert and officials can cancel it. At its time it is published once, with delivery records, push and SMS as for an immediate alert. A cancelled alert is never published.

FR-35 Bulk assignment. Assignable reports have a checkbox. With 1–20 selected from one department, one field worker can be assigned to all of them. Each report has its own transaction; any that cannot be assigned are listed with the reason and do not block the rest.

FR-36 Transparency page. Anyone can open it without signing in. It shows open issues, issues fixed in 30 days, average time to fix, SLA compliance per department, reports by category, open issues by area and the average rating, with no names, descriptions, photos or exact locations.

FR-38 Public home. A signed-out visitor lands on a page listing active alerts (emergencies first) and open issues on a map with approximate locations; choosing an area filters both. No names, descriptions, photos or exact locations appear. Signed-in users still go to their own home.

FR-39 Guest reporting. "Report an issue" opens the form without an account, and the report appears under My reports on that device with a banner offering an account. Creating an account (email/password or Google) keeps the same reports. Signing in to an existing account moves them, and the code works once. A guest's fourth report in an hour, an eleventh guest session from one address in an hour, and guest SMS opt-in or profile changes are refused. Signing out as a guest is explained first.

FR-37 Service rating. The resident may give 1–5 stars and a comment of up to 300 characters when confirming a fix. Other values are refused. Staff see the rating on the report, and the public page shows the average.

FR-29 Google sign-in. A first-time Google user is taken to a profile step that needs a name and a suburb, and always becomes a Resident. An existing profile cannot be re-created.

FR-30 Password reset. The same confirmation is shown whether or not the email is registered.

FR-31 Profile. An invalid SA phone number is refused. A resident who changes suburb receives that suburb's alerts. Staff have no suburb.

FR-32 Account deletion. It requires typing DELETE. Afterwards the user cannot sign in. Their reports show "Former resident" without photos and generate no notifications. Staff cannot self-delete.

5. SYSTEM DESIGN SPECIFICATION

This section describes the design as it was finally built. All diagrams were drawn in PlantUML, are stored in docs/diagrams/, and use the same names as the code and the requirements.

5.1 Use Cases and Workflows

The use-case diagram (Figure 2) shows the four user roles, the public, and the scheduler that runs SLA escalation, automatic closure and scheduled alerts. The Administrator can do everything an Official can.

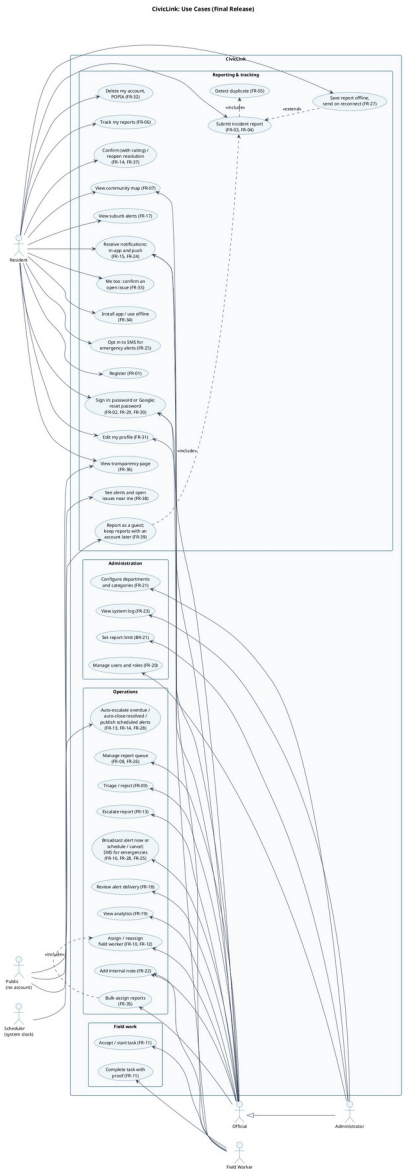


Figure 2: Use-case diagram of the final release

I picked two end-to-end workflows from the core of the system. For each one the diagrams show the validation, permission, conflict and alternative paths as well as the normal path.

Workflow W1: Submit an incident report (FR-03, FR-04, FR-05)

The resident fills in the three-step form, which checks the input as they go. Photos are compressed in the browser (to at most 1280 px and 700 KB) and sent with the request. If the resident is not signed in, the app first starts a guest session (FR-39), and if there is no connection the report is saved on the phone and sent later with the same `clientId` (FR-27). On the server, `submitReport` checks the caller and the input, checks each photo's real file type from

its first bytes, and then runs transaction TX-1. TX-1 makes sure the category is still active, looks for open reports of the same category within 100 m, takes the next reference number and saves either a new report (with its public map entry and notifications to the department's officials) or a duplicate linked to the original.

W1: Submit incident report (FR-03, FR-04, FR-05, FR-27, FR-39, TX-1)

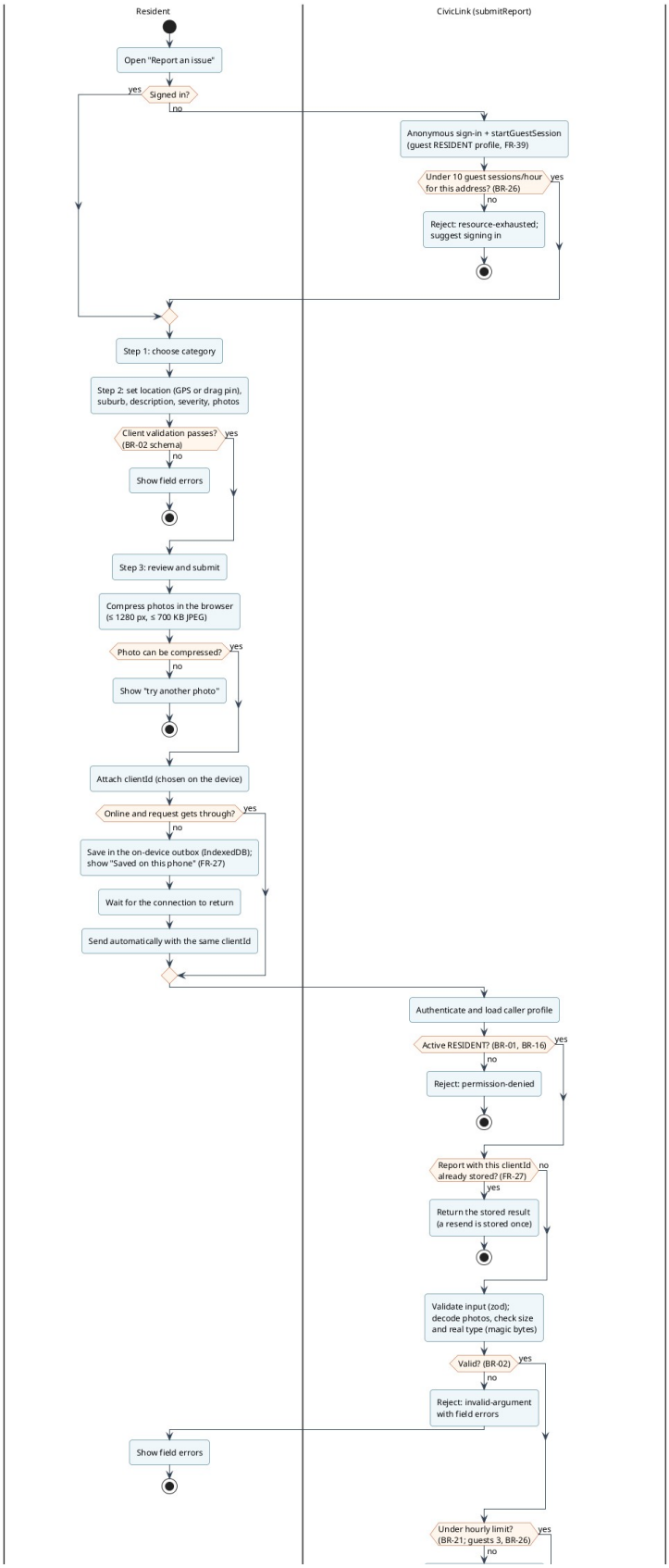


Figure 3: Activity diagram, W1 submit incident report

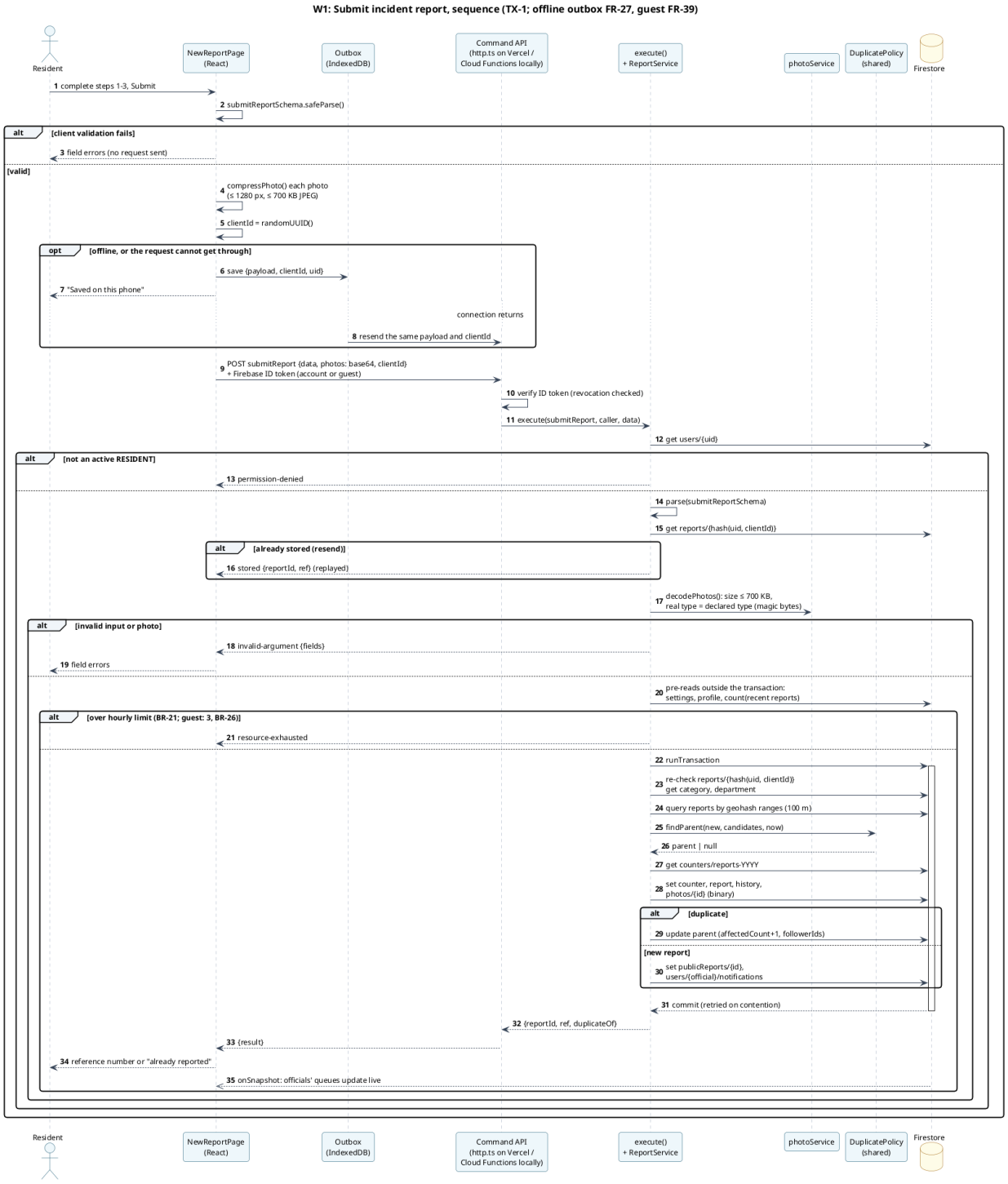


Figure 4: Sequence diagram, W1 submit incident report

Workflow W2: Triage, assign, complete and confirm (FR-09 to FR-14)

An official reviews or rejects a report and then assigns it to a field worker in transaction TX-2. The field worker accepts, starts and completes the task, and transaction TX-3 marks the report

as resolved. Finally the resident confirms the fix or reopens the report. The diagrams also show what happens when a report is rejected, when it already has an active task, when the worker is not eligible, when task steps are skipped, when the proof photo is missing, when the task was reassigned while the worker was on site, and when the resident reopens it.

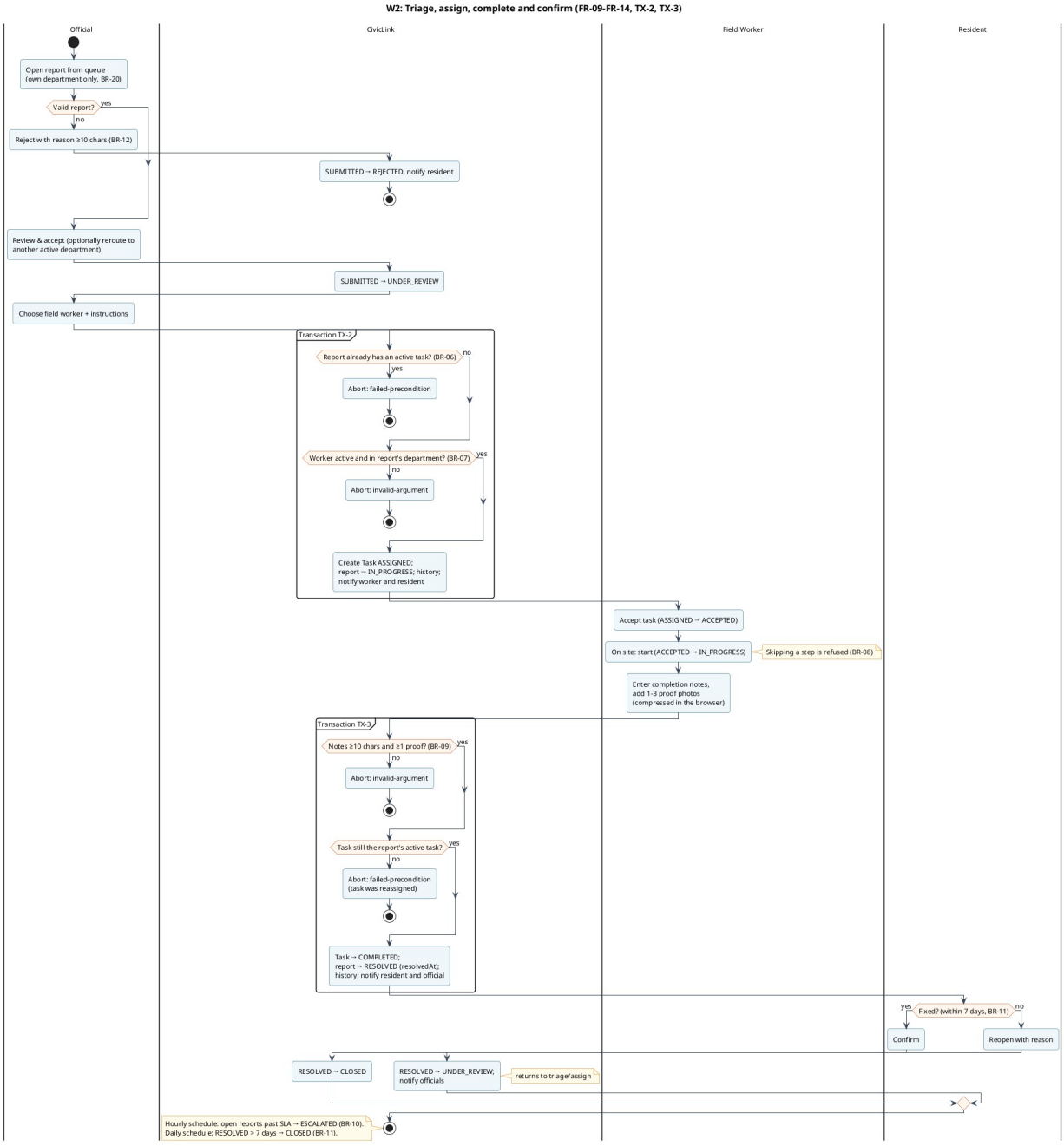


Figure 5: Activity diagram, W2 triage, assign, complete and confirm

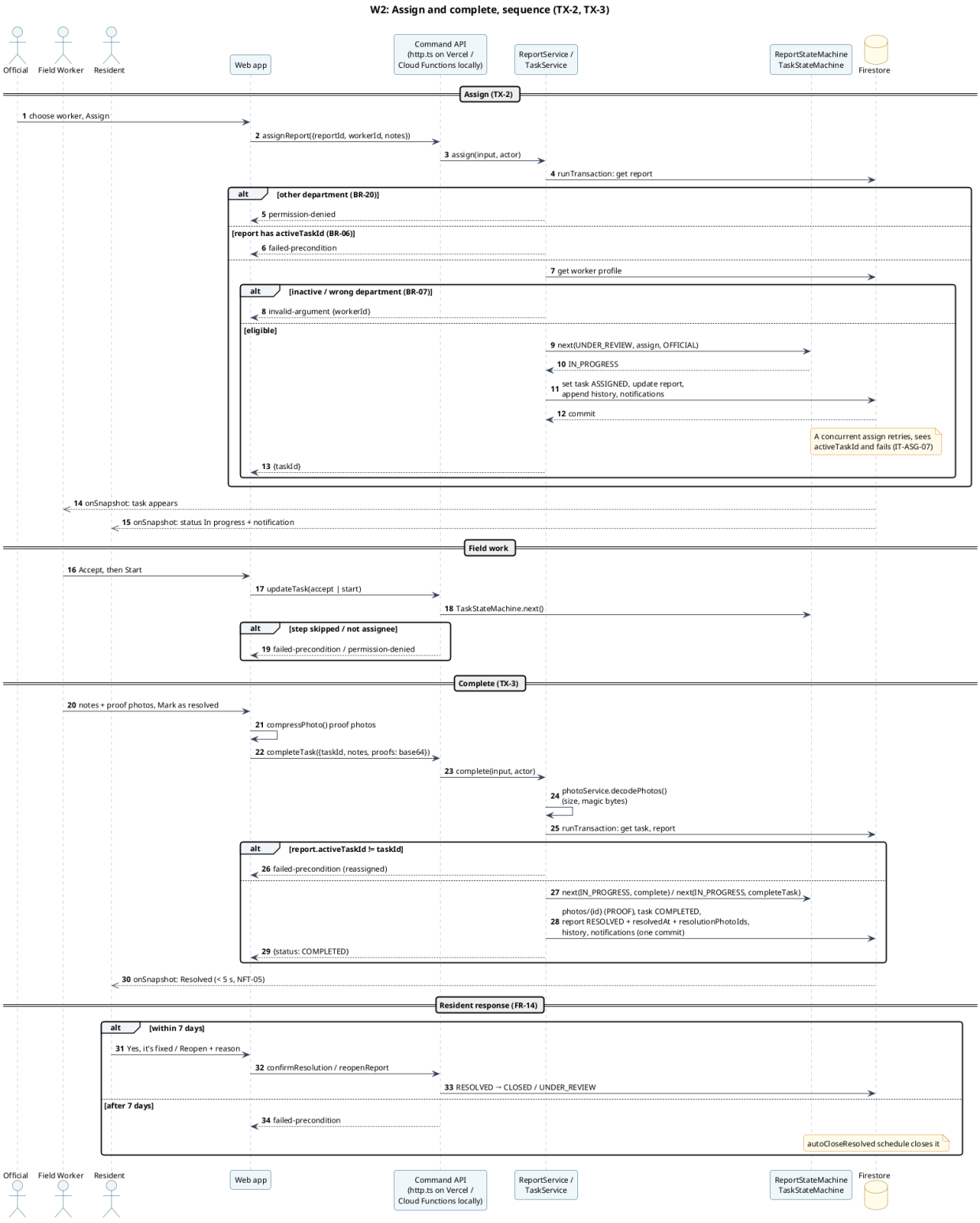


Figure 6: Sequence diagram, W2 assign and complete

Entity lifecycles

The **Report** goes through eight states (Figure 7). These are implemented once in ReportStateMachine in the shared package, which the server uses to enforce the transitions and the interface uses to show only the actions that are allowed. The **Task** lifecycle (Figure 8) is implemented in TaskStateMachine.

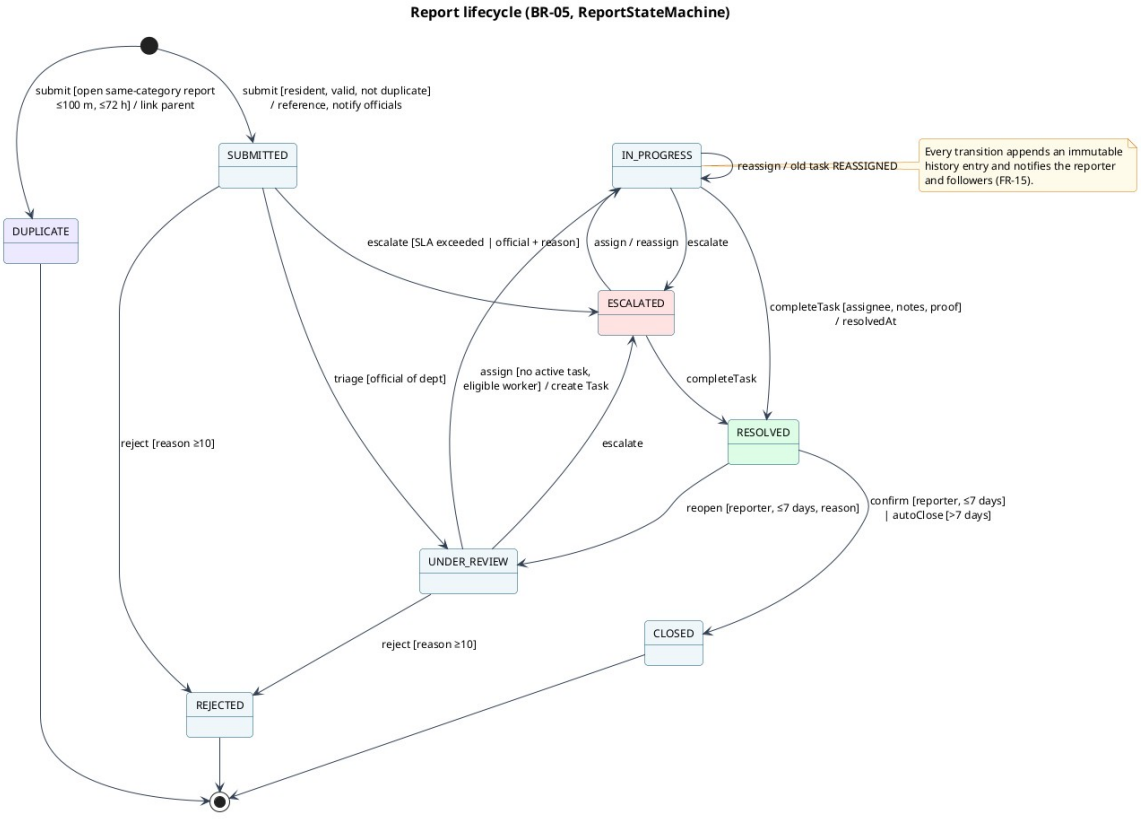


Figure 7: State diagram of the Report entity

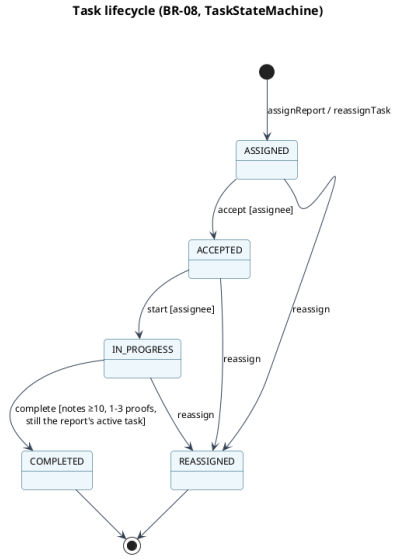


Figure 8: State diagram of the Task entity

5.2 System Architecture and Technology Stack

The architecture (Figure 9) separates reading data from changing it:

- **Reads** come straight from Firestore through real-time listeners, so screens update by themselves. Security Rules decide what each user may read.
- **Changes** always go through a server command. The command checks who the caller is and what role they have, validates the input with the shared schemas and runs the business logic in a Firestore transaction.
- **Domain logic** (schemas, state machines and policies) is written once in a shared TypeScript package used by both the browser and the server.

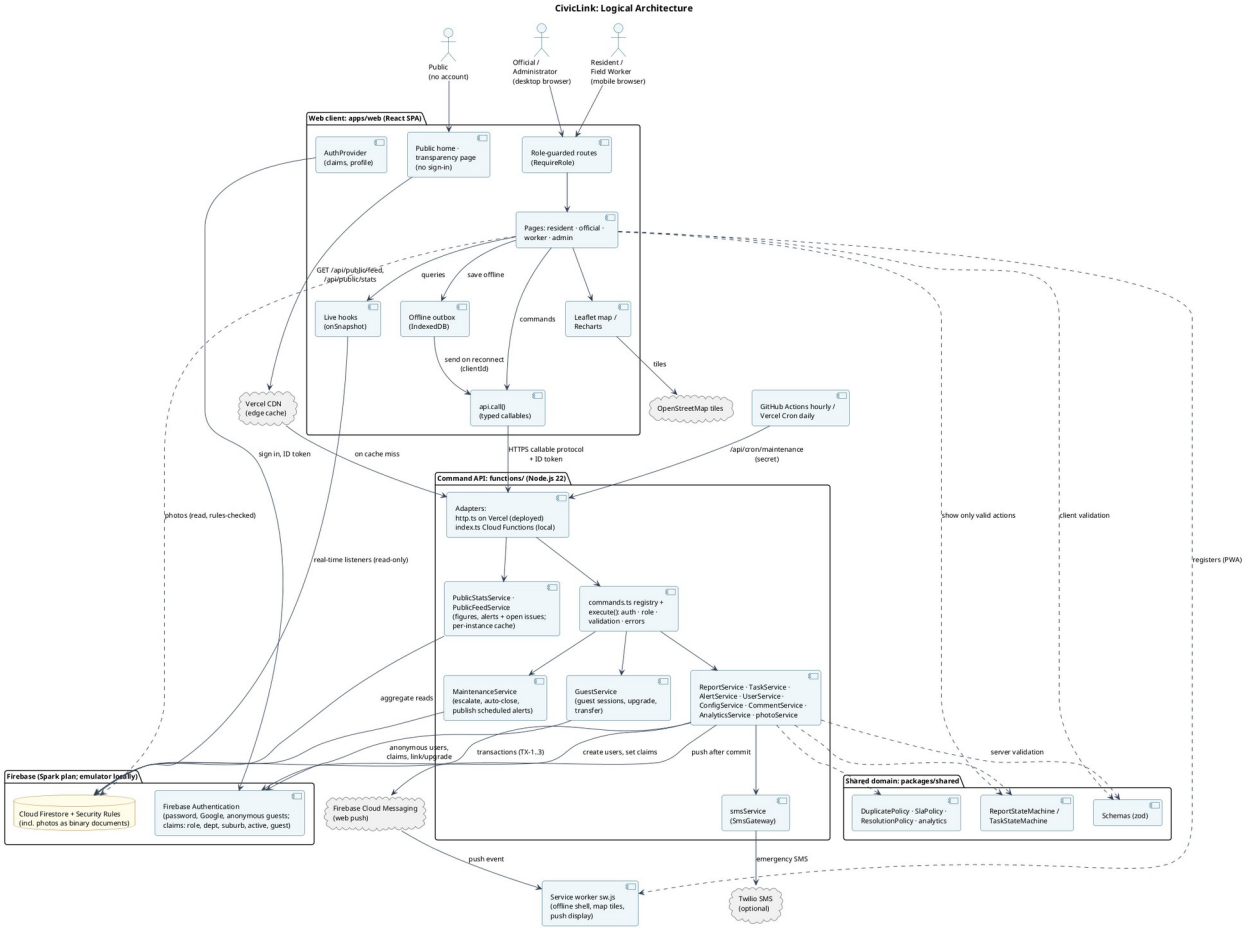


Figure 9: Logical architecture

Table 8: Component responsibilities and technology stack

Component	Technology	Responsibility
Web application	React 19 (Meta Platforms, 2026), TypeScript, Vite, React Router, Tailwind CSS	Role-specific user interfaces, client-side validation, real-time views
Maps	Leaflet 1.9 (Leaflet, 2023) with OpenStreetMap tiles (OpenStreetMap contributors, 2026)	Location capture (GPS or draggable pin) and incident maps
Charts	Recharts 3	Analytics visualisation
Command API	Node.js 22 command registry with two adapters: Vercel serverless functions (deployed) and Cloud Functions (local emulator)	Authentication and role checks, validation, business rules, transactions, notifications
Scheduled jobs	GitHub Actions (hourly) and Vercel Cron (daily) calling a secret-protected endpoint, plus a background check after API requests; Cloud Functions scheduler locally	SLA escalation, auto-close, publishing scheduled alerts
Notifications	Firebase Cloud Messaging web push (Firebase, 2026c) via the app's service worker; Twilio Programmable Messaging REST API for SMS (Twilio, 2026a)	Push to devices; SMS for emergency alerts
Offline and caching	Web app manifest (W3C, 2023) and service worker (app shell, map tiles), IndexedDB outbox, Firestore offline cache; Vercel CDN edge cache (Vercel, 2026a)	Installable app, offline drafts, fast repeat visits, cached public figures
Shared domain package	TypeScript, Zod 4 (Zod, 2026)	Schemas, report/task state machines, duplicate, SLA and resolution policies, analytics
Database	Cloud Firestore (Firebase, 2026b)	Persistent documents, real-time listeners, transactions
Photo storage	Cloud Firestore binary documents (photos)	Report and proof photos, compressed in the browser (decision D2)
Authentication	Firebase Authentication with custom claims; anonymous sign-in for guests (Firebase, 2026a)	Email/password, Google and guest sign-in, role, department, suburb and active-account claims
Access control	Firestore Security Rules (Firebase, 2026e)	Server-side enforcement of the permission matrix
Runtime: deployed	Vercel (static site + Node serverless functions, Cape Town region; Vercel, 2026b) and Firebase Spark (Auth, Firestore)	Free-tier production configuration (decision D2)
Runtime: local	Firebase Emulator Suite (Firebase, 2026d)	Local Auth, Firestore and Functions for development, tests and offline

Component	Technology	Responsibility
		demonstration
Testing / CI	Vitest, @firebase/rules-unit-testing, Playwright (Microsoft, 2026), axe-core, GitHub Actions, gitleaks	Automated verification
Browser hardening	Content Security Policy (MDN contributors, 2026), HSTS and related headers in vercel.json	Defence against script injection and framing

5.3 Key Architecture and Design Decisions

Four decisions shaped the system more than any others, so I describe them in full. Three smaller ones are summarised in the table after them. Each decision also has an architecture decision record in docs/adr/.

D1: Every change goes through a server-side command (ADR-0001).

- *Context:* the most important rules involve several documents at once: taking the next reference number, the 100 m duplicate search, allowing only one active task per report, and resolving the report when its task is completed.
- *Options:* (a) let the app write to Firestore directly and protect it with Security Rules only; (b) send every change through a callable function that runs a transaction, and let the rules allow reads only; (c) write a separate Node/Express API with PostgreSQL.
- *Decision:* (b).
- *Rationale:* Security Rules cannot check rules that span several documents in one transaction, so option (a) could not enforce BR-03, BR-04 or BR-06. Option (b) gives me one place where every rule is enforced, while the app still gets real-time reads from Firestore. Option (c) would have meant rebuilding authentication, real-time updates and storage, and throwing away three phases of design.
- *Consequences:* the services are easy to test and concurrency is handled by Firestore transactions (two officials assigning the same report at once: exactly one succeeds). The downside is that Firestore has no foreign keys, so integrity depends on the transactions and the rules (Section 5.6), and deploying Cloud Functions needs Firebase's paid plan, which led to D2.

D2: Free-tier deployment: Firebase Spark, a Node API on Vercel, and photos in Firestore (ADR-0006).

- *Context:* I wanted a live deployment that costs nothing, but Cloud Functions and Cloud Storage both need Firebase's paid Blaze plan.
- *Options:* (a) Blaze with a budget alert; (b) move to Supabase; (c) stay on the free Spark plan (Authentication and Firestore), run the command API on a free Node host, and store photos in Firestore.
- *Decision:* (c). The API runs as Vercel serverless functions next to the web app, and photos are compressed in the browser to at most 700 KB and stored as binary documents.
- *Rationale:* my commands did not depend on how they were called, so a second small adapter could reuse every service unchanged and speak the same callable protocol; the app only changes the URL it calls. Vercel's free functions do not go to sleep, which matters during a live demo. Moving to Supabase would have meant rewriting the back end and about 90 tests.
- *Consequences:* the two adapters must behave the same, so CI runs the whole end-to-end suite through both. Photos are limited by Firestore's 1 MiB document size. Vercel's free cron only runs daily, so the hourly SLA job is triggered by GitHub Actions instead. The emulator setup is still used for development, tests and an offline demo.

D3: Offline reports through an on-device outbox and idempotent commands (ADR-0007).

- *Context:* residents with poor signal must be able to report without a connection (FR-27). Because reports are created by a server command (D1), Firestore's own offline write queue does not help.
- *Options:* (a) the Background Sync API in the service worker; (b) an outbox in IndexedDB that the app sends when the browser is back online; (c) keep reporting disabled offline, as in the Phase 3 baseline.
- *Decision:* (b), with a `clientId` created on the phone and sent with the report. The server builds the report's document id from the user and this `clientId`, so if the same report arrives twice (for example because the response was lost), it returns the report it already has instead of creating a second one. This makes the command idempotent.
- *Rationale:* Background Sync does not work in Firefox or Safari (NFR-09). The outbox works in every browser, keeps the compressed photos, and belongs to one account. Because the command is idempotent, it is safe for the app to keep retrying until it gets an answer.

- *Consequences:* a saved report only gets its reference number once it is sent. If the server refuses a saved report, it stays on the phone with the reason, so nothing disappears silently.

D4: Guests through anonymous sign-in, not open endpoints (ADR-0009).

- *Context:* people should be able to report without creating an account (FR-39), but every change must still be authorised on the server (D1, NFR-01), and I had to limit abuse.
- *Options:* (a) an unauthenticated "submit" endpoint; (b) Firebase anonymous sign-in plus a guest Resident profile created by the server; (c) keep registration compulsory.
- *Decision:* (b).
- *Rationale:* a guest is a real Firebase user (Firebase, 2026a), so every existing command check and Security Rule applies without changes, and the guest can follow and confirm their own report. Later the guest can create an account by linking an email/password or Google sign-in to the same user, so nothing has to be copied. Option (a) would have needed a second, weaker authorisation path.
- *Consequences:* guest sessions cost nothing to create, so BR-26 limits guests to 3 reports an hour and 10 new guest sessions per network address per hour. A guest who clears their browser loses access unless they made an account, so the app warns them before they sign out.

Table 9: Supporting decisions

Decision	Reason	Record
React single-page application with Vite, not Next.js	All data comes from the Firebase SDK in the browser, so server rendering adds complexity without benefit; a static build deploys to any CDN	ADR-0002
Leaflet with OpenStreetMap, not Google Maps	No API key or billing; the suburb that drives alert targeting is chosen from a list, so reverse geocoding is not needed (officials rely on the pin and landmark)	ADR-0004
Public figures and the public feed served from the CDN cache, not public Firestore reads	No Security Rule opens data to anonymous users and only aggregates or the anonymised map projection leave the server; database reads stay bounded however many people visit (figures ≤ 5 min old, feed ≤ 1 min)	ADR-0008

5.4 Application Class Design

Figure 10 shows the implemented classes, split into two parts so it is readable (the full diagram is docs/diagrams/class.png). The classes fall into three groups:

- **Domain entities** (packages/shared/src/models.ts), stored as Firestore documents.

- **Domain logic** (packages/shared/src): ReportStateMachine and TaskStateMachine define the allowed transitions; DuplicatePolicy (100 m, 72 h), SlaPolicy and ResolutionPolicy (7 days) are pure, fully unit-tested rules.
- **Application services** (functions/src/services), described below.

This is how the services fit together:

- **Entry point:** every callable is wrapped by command(), which authenticates the caller, loads their profile (refusing deactivated accounts), checks the role, validates the input and maps errors.
- **Report and task services:** ReportService implements submission (TX-1, idempotent with a client id), triage, rejection, escalation, assignment (TX-2) and bulk assignment, reassignment, confirmation with rating, reopening and "Me too". TaskService implements the field-worker steps and completion (TX-3).
- **Single path for status changes:** both services use ReportLifecycle.applyTransition, so every status change is validated by the state machine, recorded in the history, reflected in the public projection and notified.
- **Other services:** AlertService (immediate and scheduled alerts), UserService, ConfigService, CommentService, AnalyticsService, PublicStatsService (anonymous figures), MaintenanceService (all scheduled rules), and the delivery helpers pushService (FCM) and smsService (a gateway interface with Twilio and simulated implementations) implement the remaining capabilities.

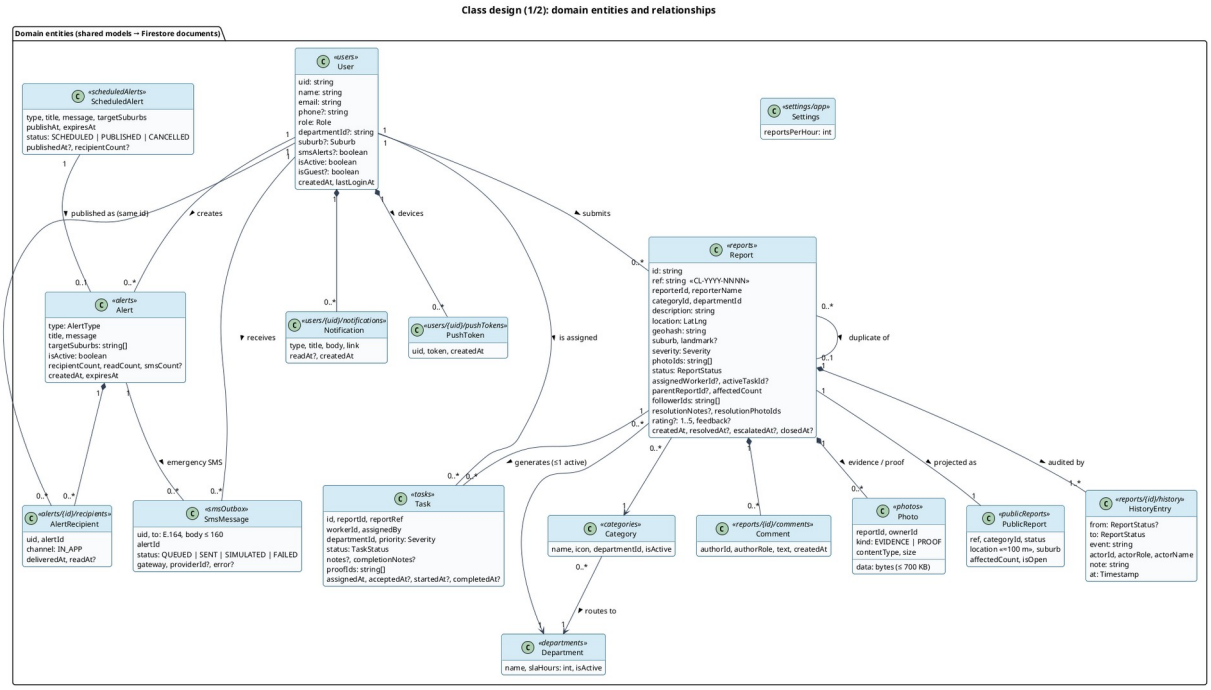


Figure 10a: Class diagram (part 1), domain entities and relationships

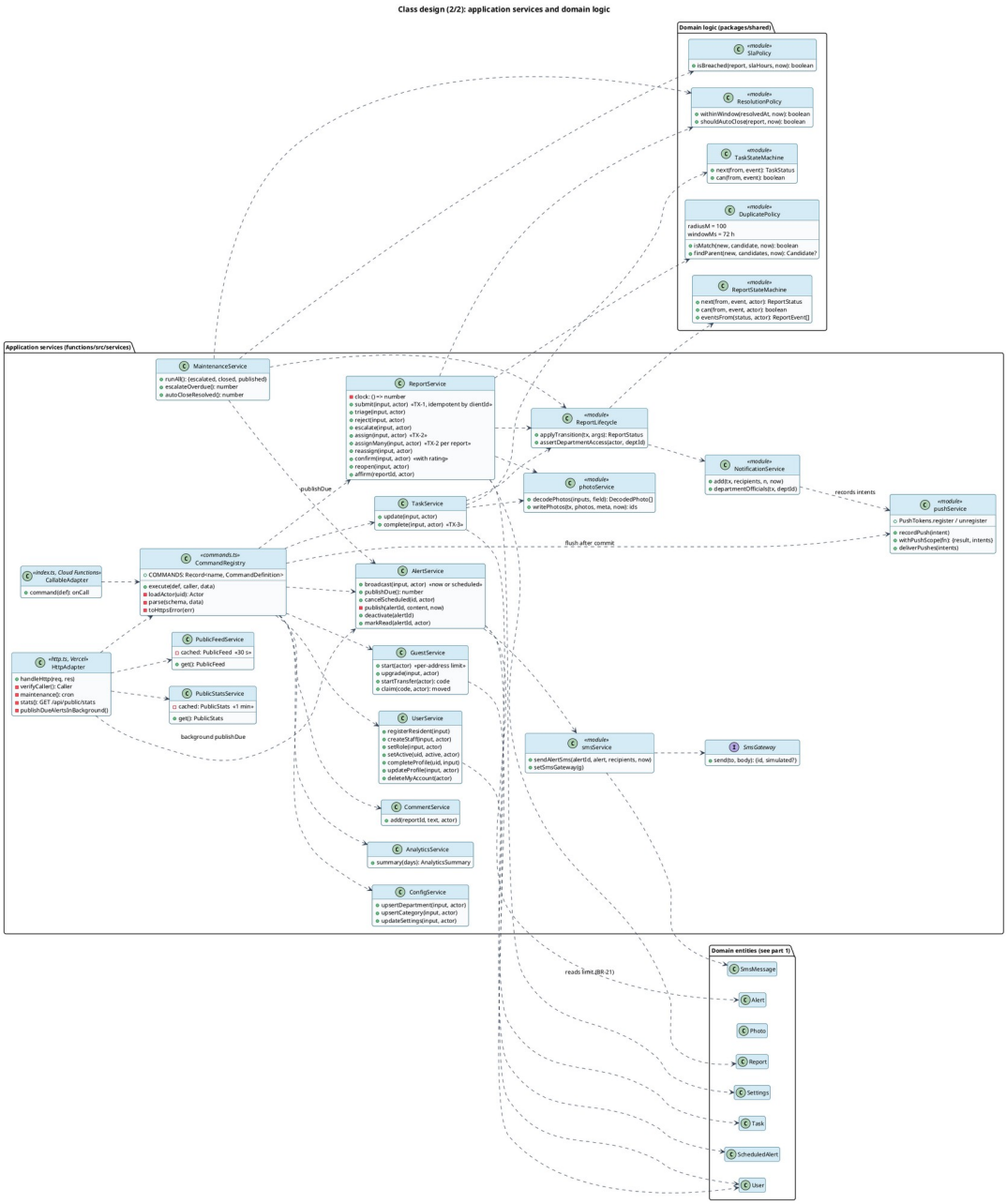


Figure 10b: Class diagram (part 2), application services and domain logic

5.5 User Interface and Interaction Design

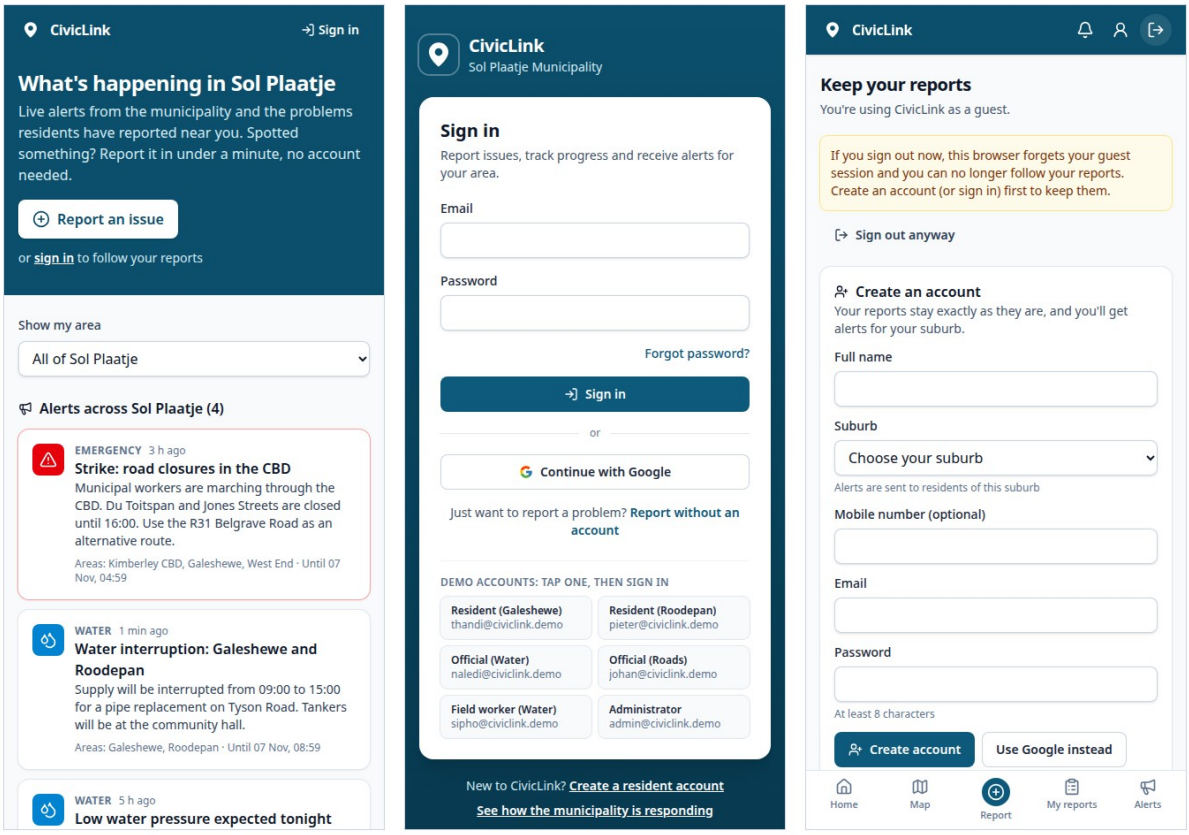
Each role has its own part of the interface:

- **Residents** use a mobile layout with a bottom navigation bar (Home, Map, Report, My reports, Alerts).
- **Field workers** use the same mobile layout, coloured green and reduced to their task list.
- **Officials and administrators** use a desktop layout with a sidebar.

Every screen has a loading, empty, error and access-denied state. The screenshots were taken from the submitted system with the seeded data, and the photos in them are sample images. Phone screens are grouped three to a figure by user journey and labelled (a) to (c); desktop screens have one figure each.

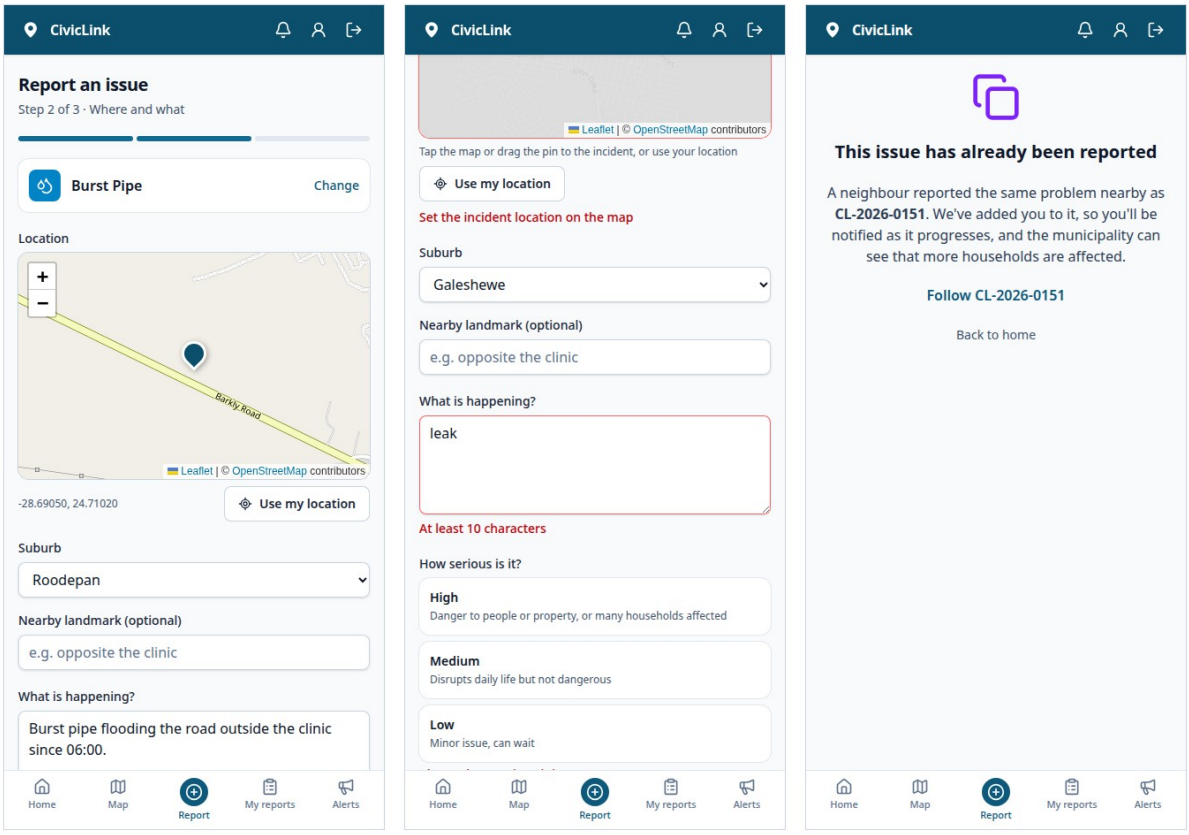
Table 10: User interface evidence

Figure	User	Purpose and main action	Requirement / workflow
11	Visitor, any user, guest	(a) Public home with alerts and open issues; (b) sign in; (c) a guest keeping their reports by creating an account or signing in	FR-38, FR-02, FR-39
12	Resident	(a) Report details with map pin, suburb, description, severity and photo; (b) validation errors before submission; (c) duplicate detected, resident follows the original	FR-03, BR-02, FR-05, W1 and its alternative paths
13	Official	Department dashboard with KPIs and reports needing action	FR-08
14	Official	Triage and assignment, with history and internal notes	FR-09, FR-10, W2
15	Official	Bulk assignment of selected reports to one field worker	FR-35
16	Official	Empty state in the report queue	FR-08
17	Field worker, resident	(a) Task completed with notes and proof photo; (b) resolved report with proof (result state); (c) confirming the fix with a rating	FR-11, FR-14, FR-37, W2
18	Official	Alert composition with a live resident preview	FR-16
19	Official	Scheduled alerts with cancel	FR-28
20	Resident	(a) Suburb alerts feed, emergency pinned first; (b) "Me too" on an open issue from the map; (c) offline, with a report saved on the phone	FR-17, FR-33, FR-27, FR-34
21	Official	Analytics computed from live data	FR-19
22	Public	Transparency page, no sign-in	FR-36
23	Administrator	User and role management	FR-20
24	Administrator	SMS outbox and test SMS	FR-25
25	Resident	(a) Permission denied on an official page (error state); (b) first Google sign-in completes the profile; (c) profile editing and POPIA account deletion	NFR-01, FR-29, FR-31, FR-32



(a) Public home (visitor) (b) Sign in (c) Guest: keep reports

Figure 11: Entry: public home, sign in, guest account



(a) Report: details (b) Validation errors (c) Duplicate detected

Figure 12: Resident reporting (W1): details, validation errors, duplicate

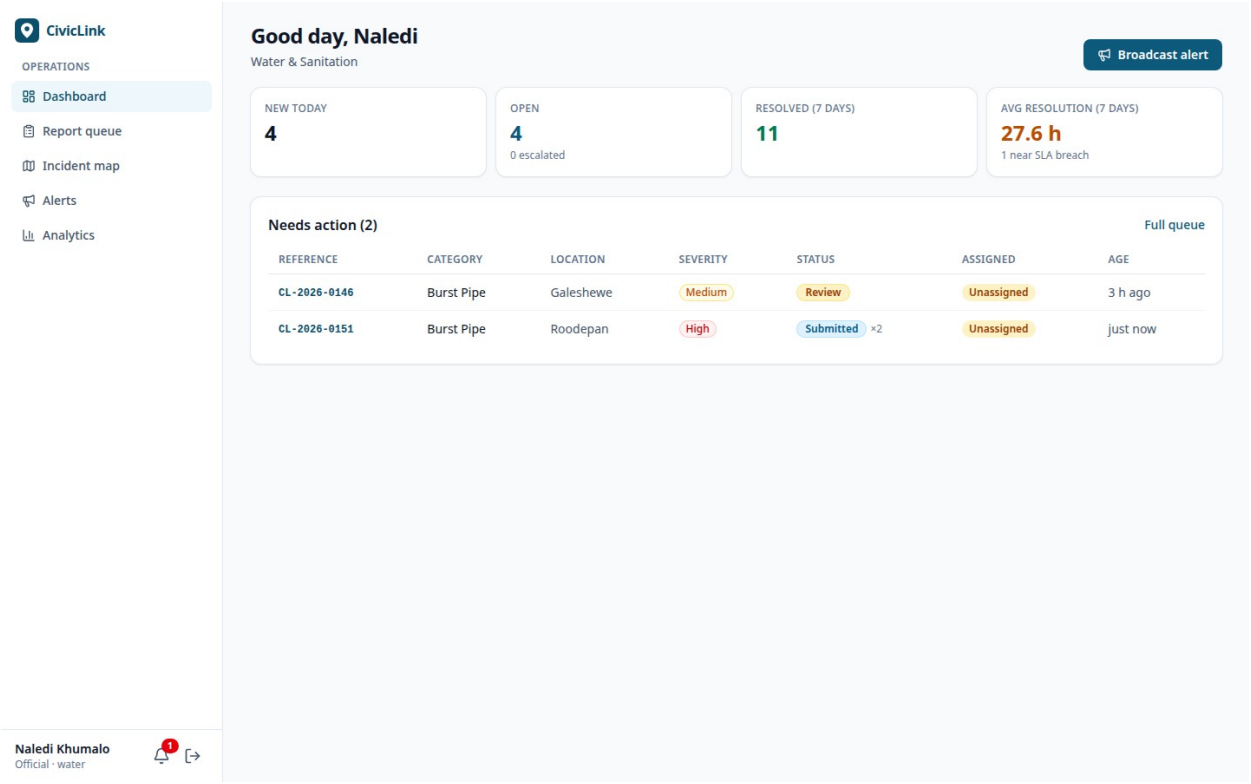


Figure 13: Official dashboard

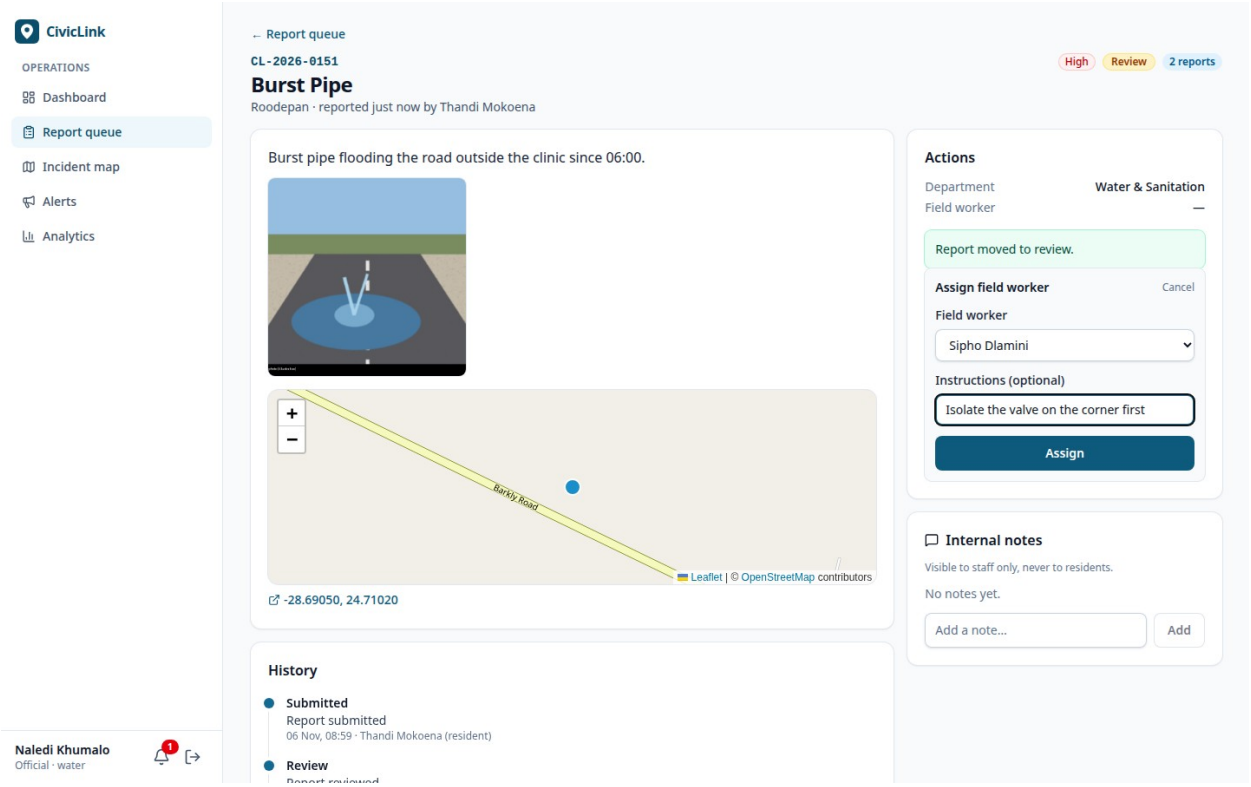


Figure 14: Official, triage and assignment

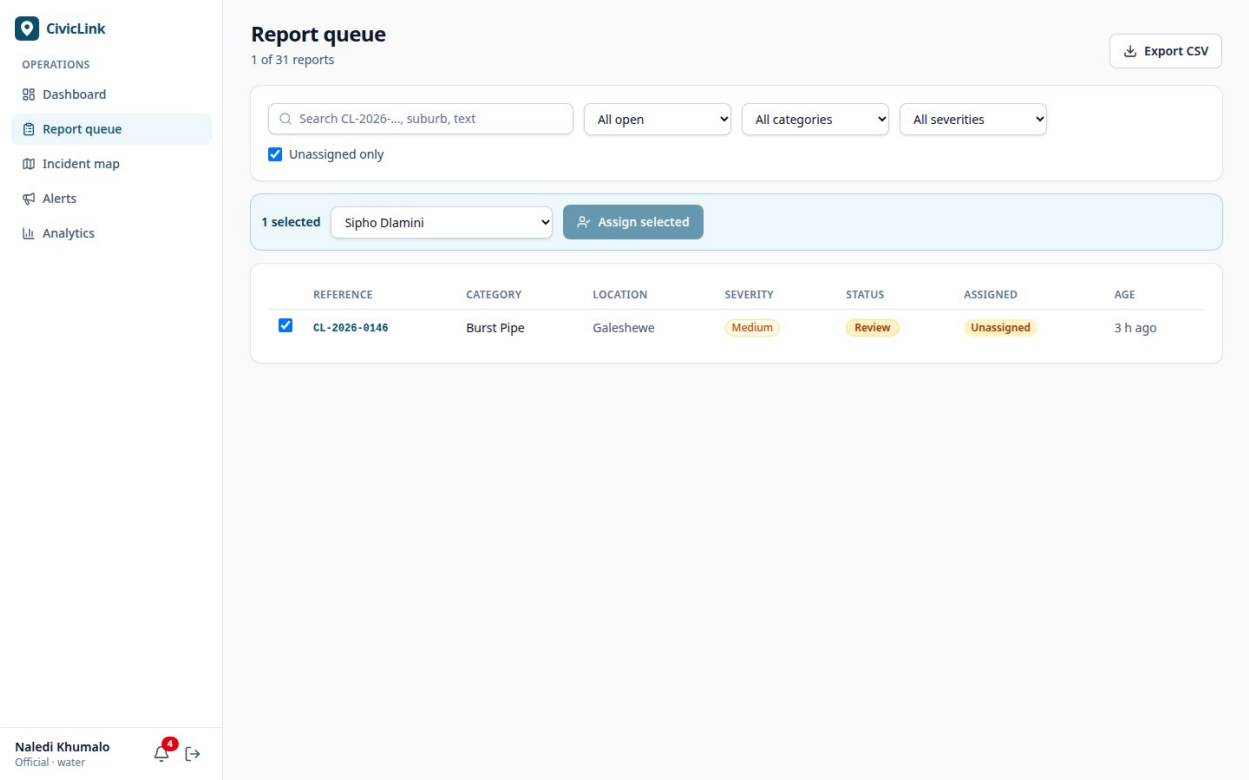


Figure 15: Official, bulk assignment

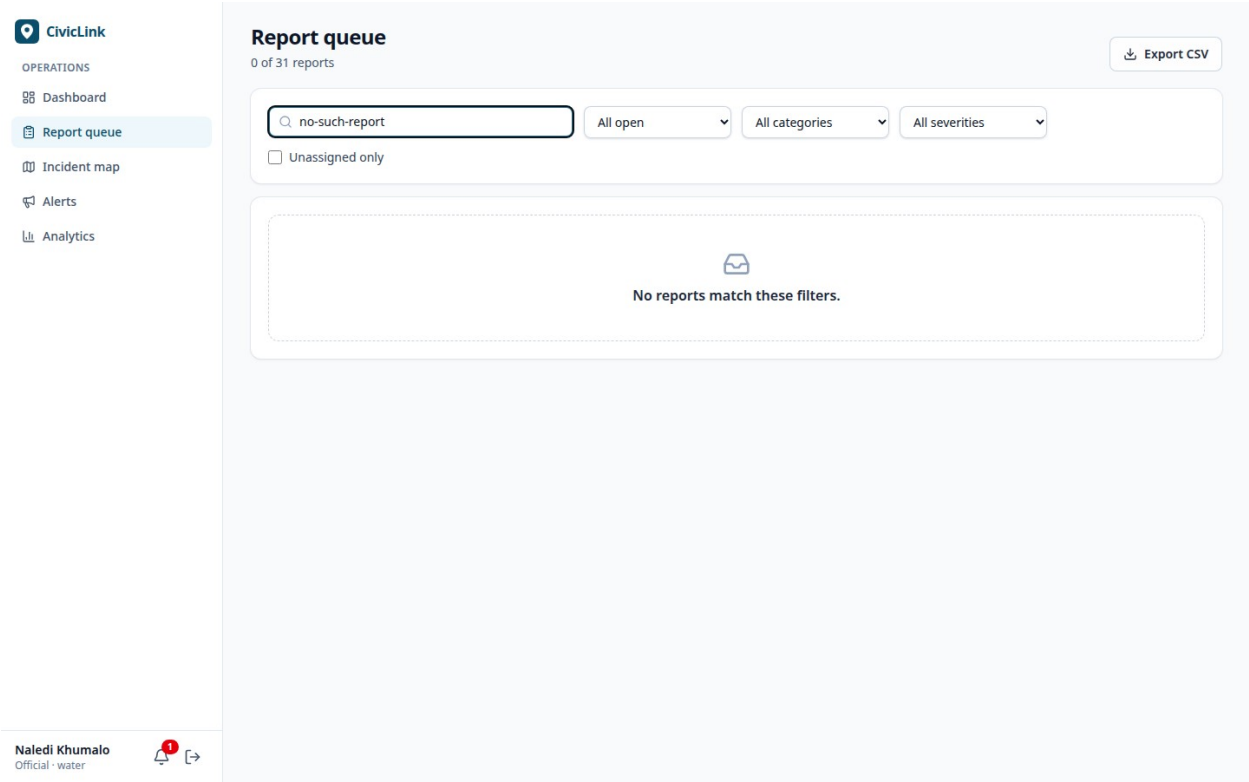
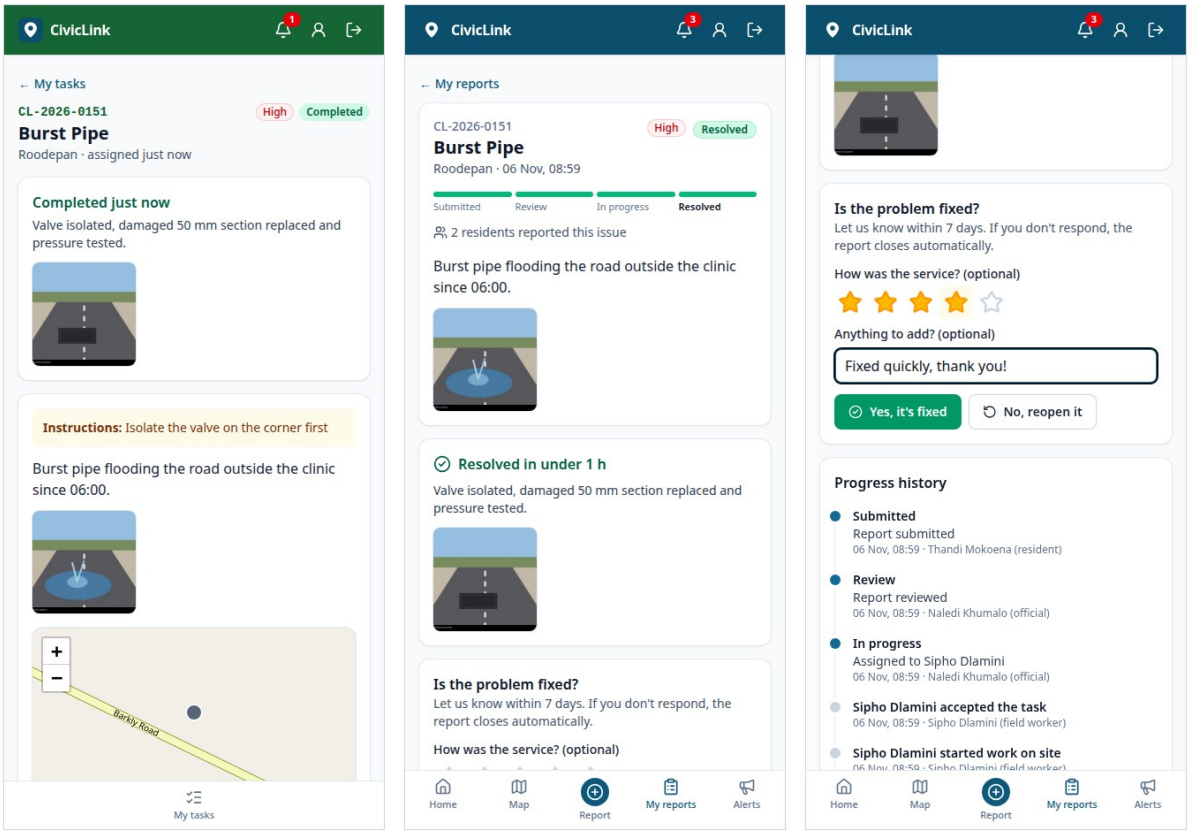


Figure 16: Empty state in the report queue



(a) Worker: completed (b) Resident: resolved (c) Resident: rating

Figure 17: Resolution (W2): worker completion, resolved report, rating

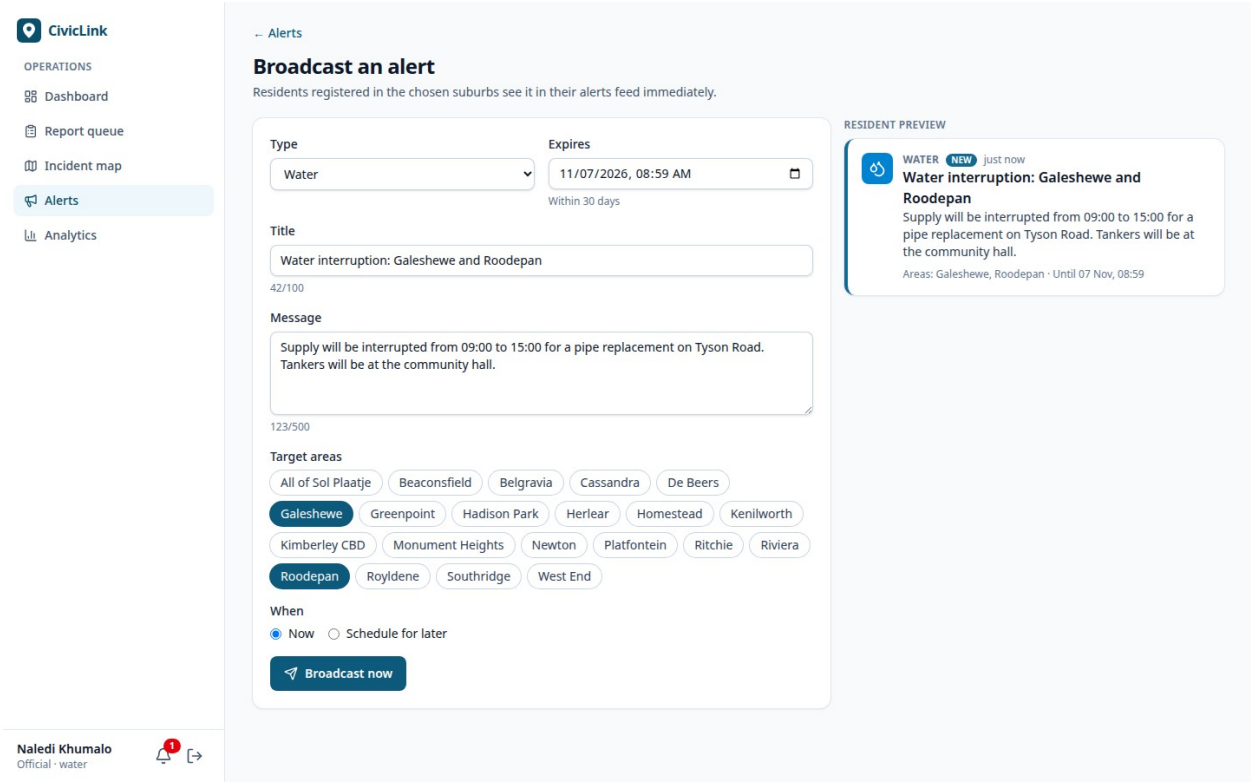


Figure 18: Official, alert composer with preview

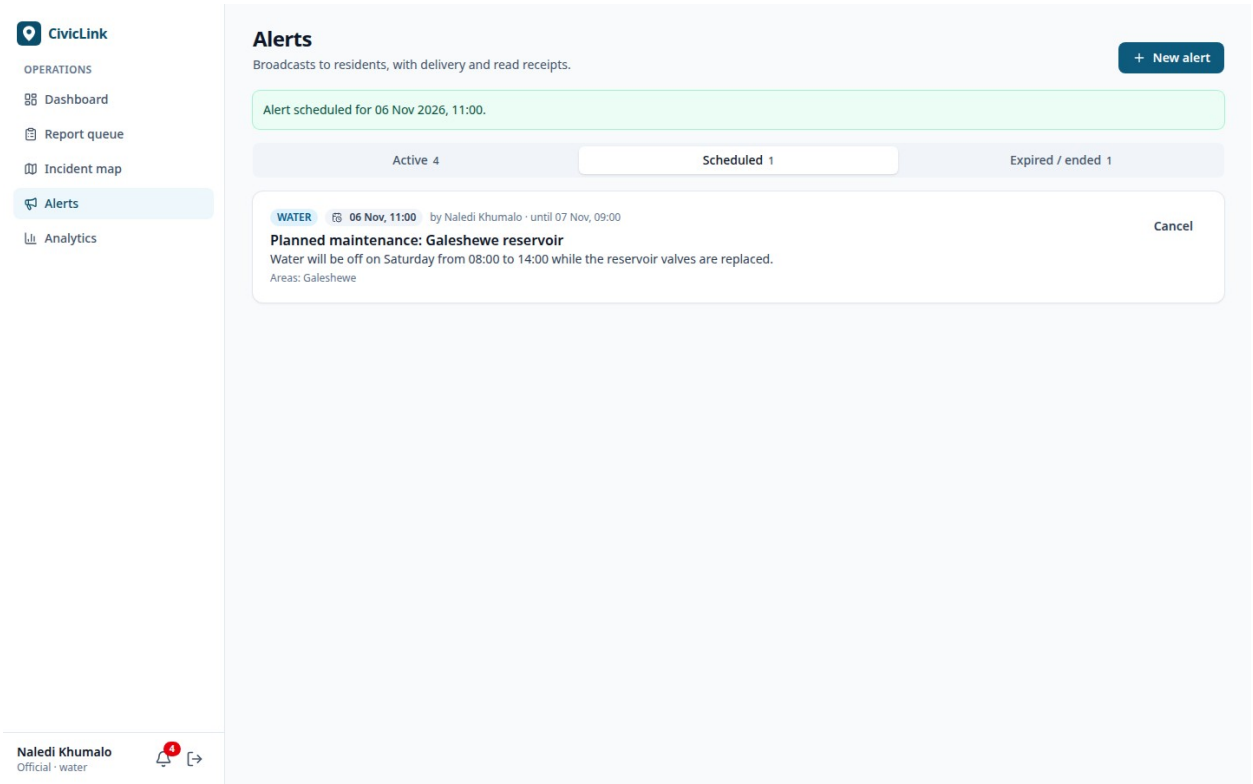
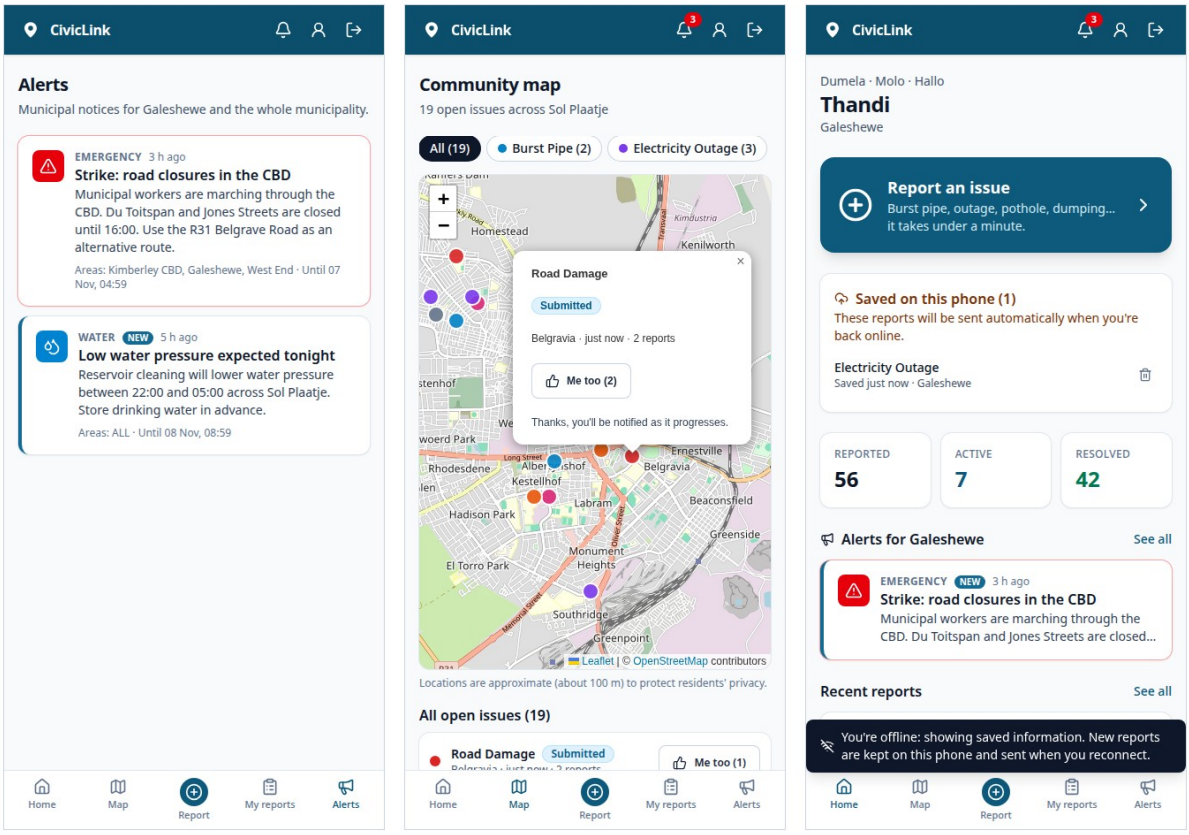


Figure 19: Official, scheduled alerts



(a) Suburb alerts (b) "Me too" on the map (c) Offline: report saved

Figure 20: Resident community features: alerts, "Me too", offline report

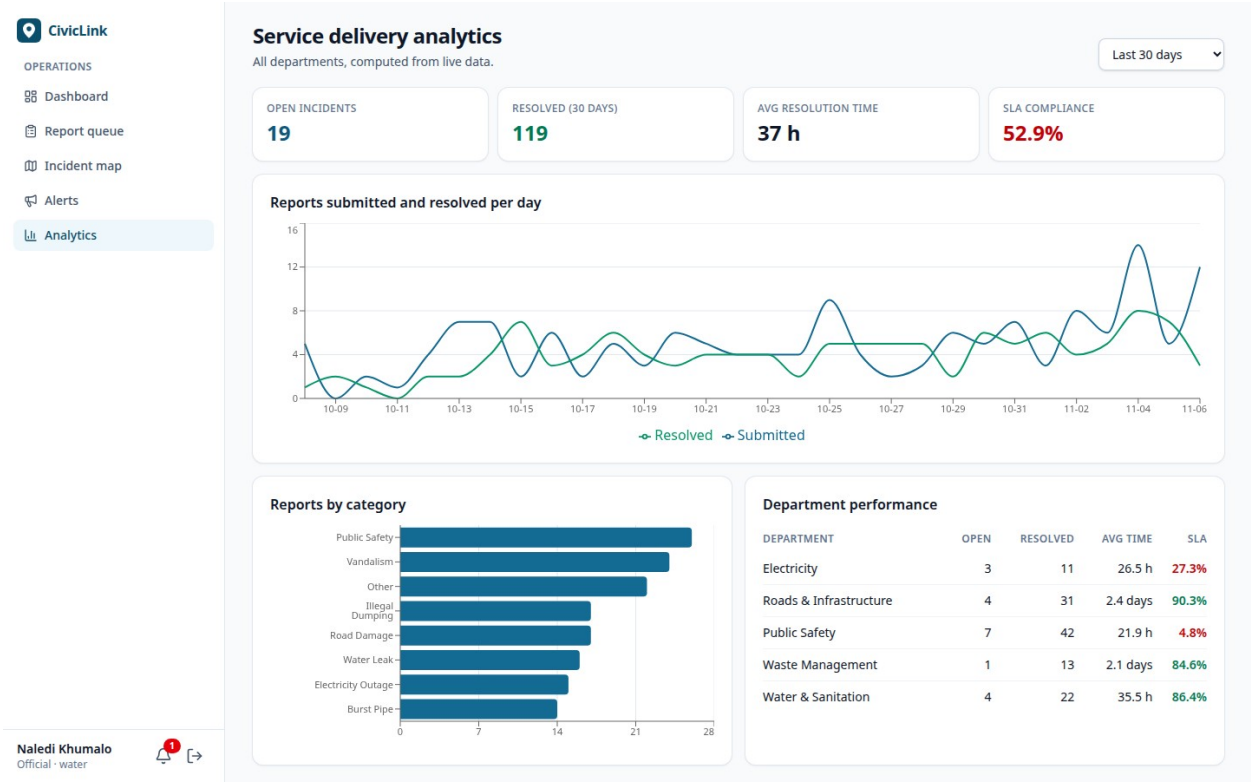


Figure 21: Service-delivery analytics

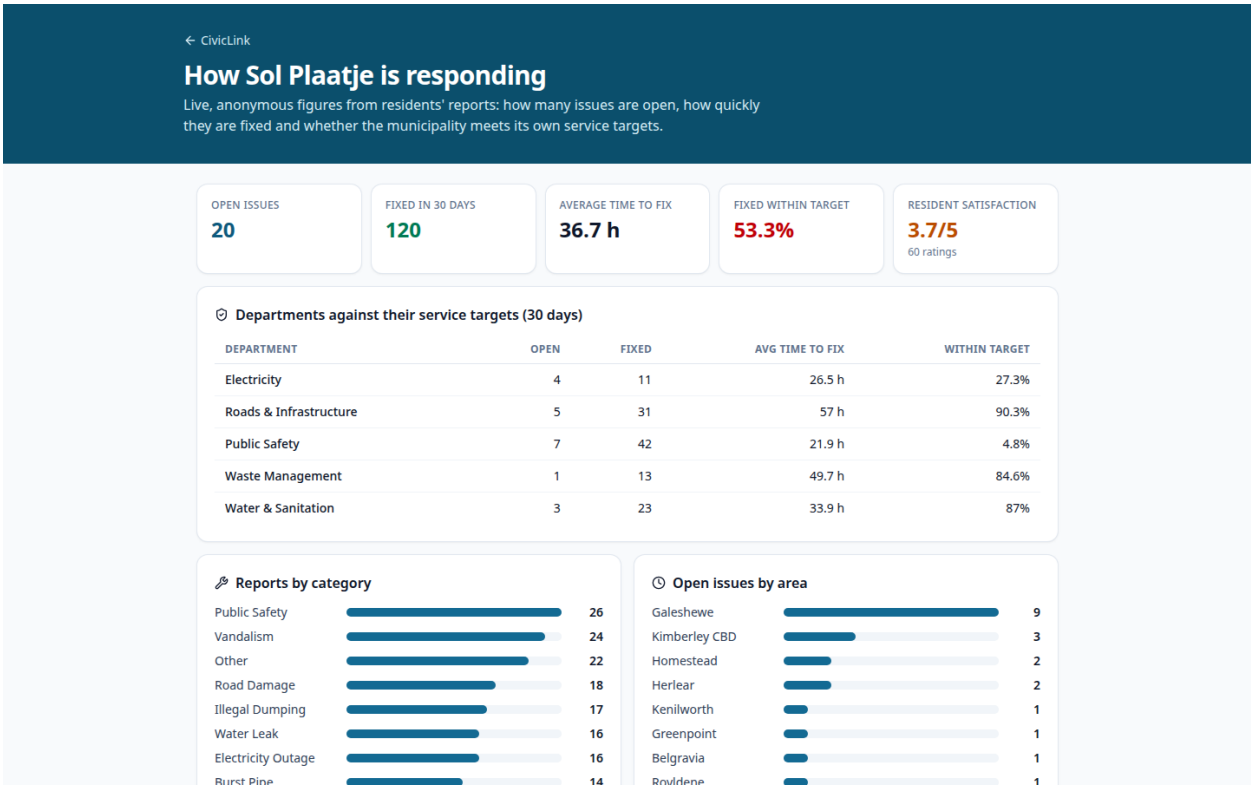


Figure 22: Public transparency page

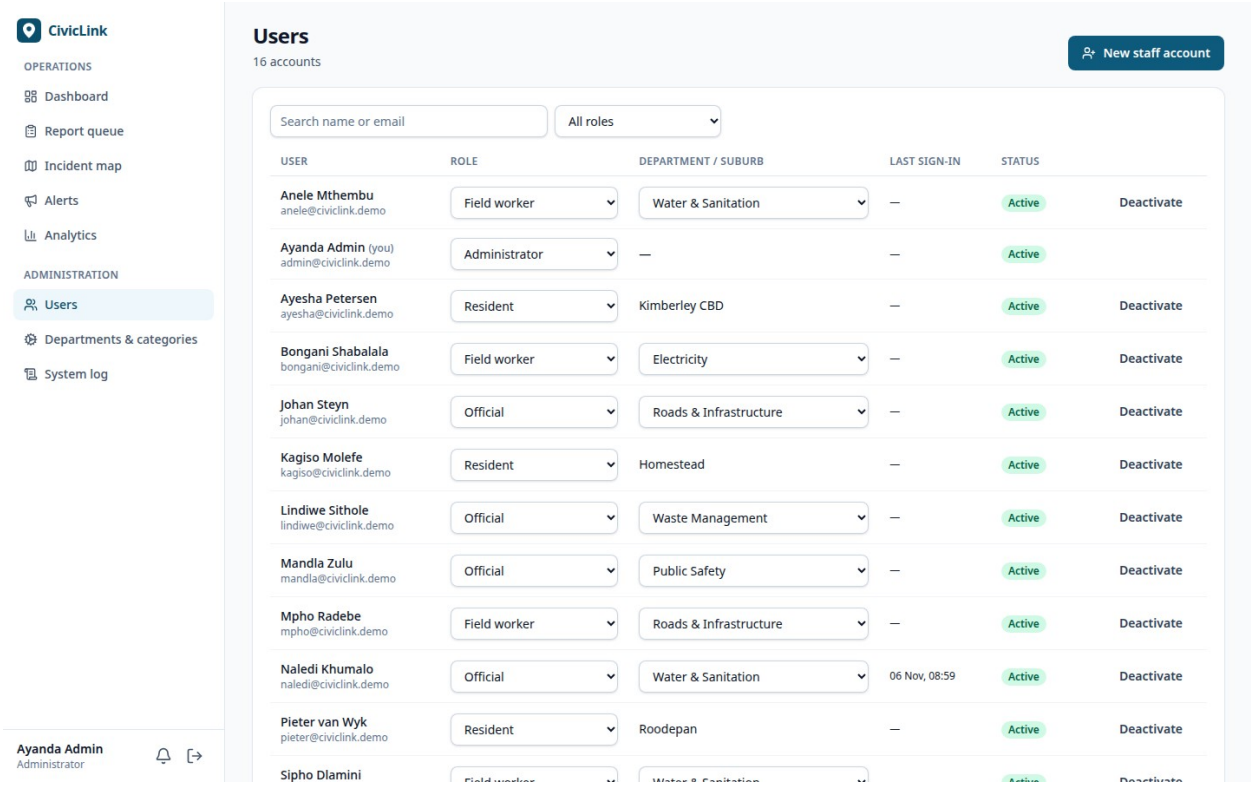


Figure 23: Administrator, user management

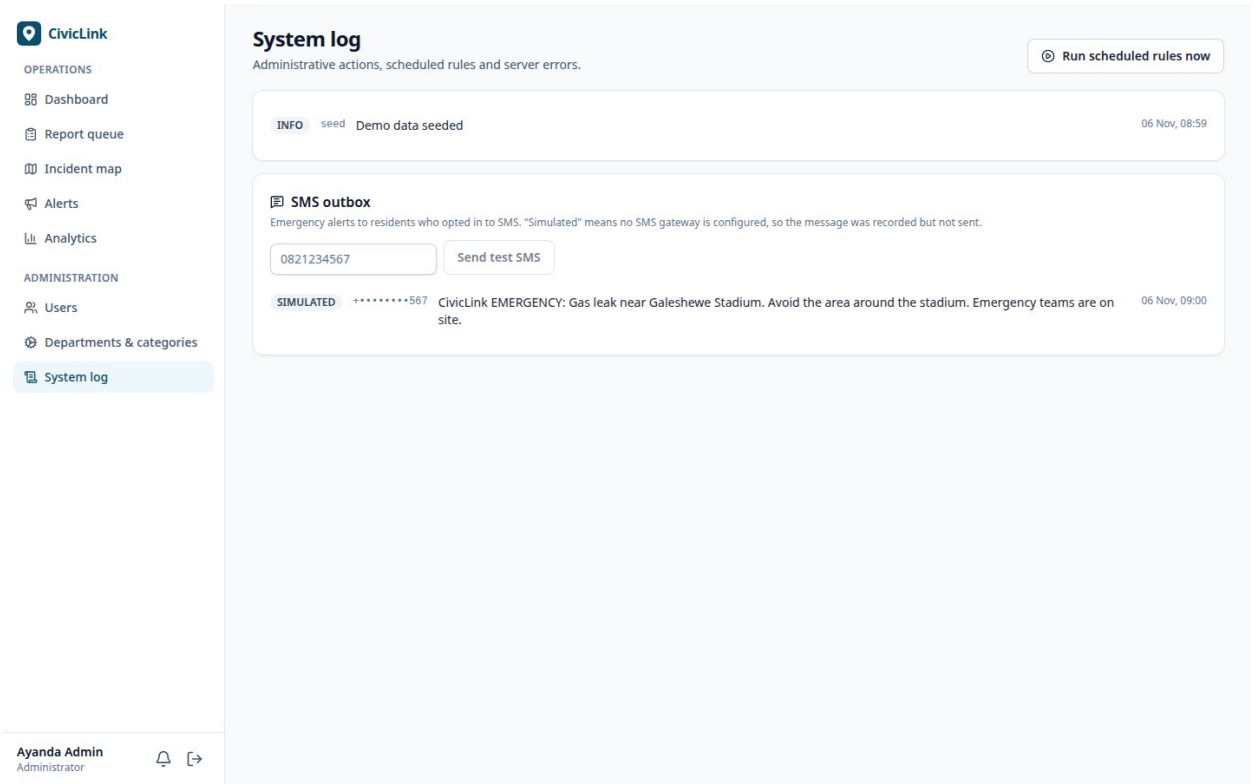
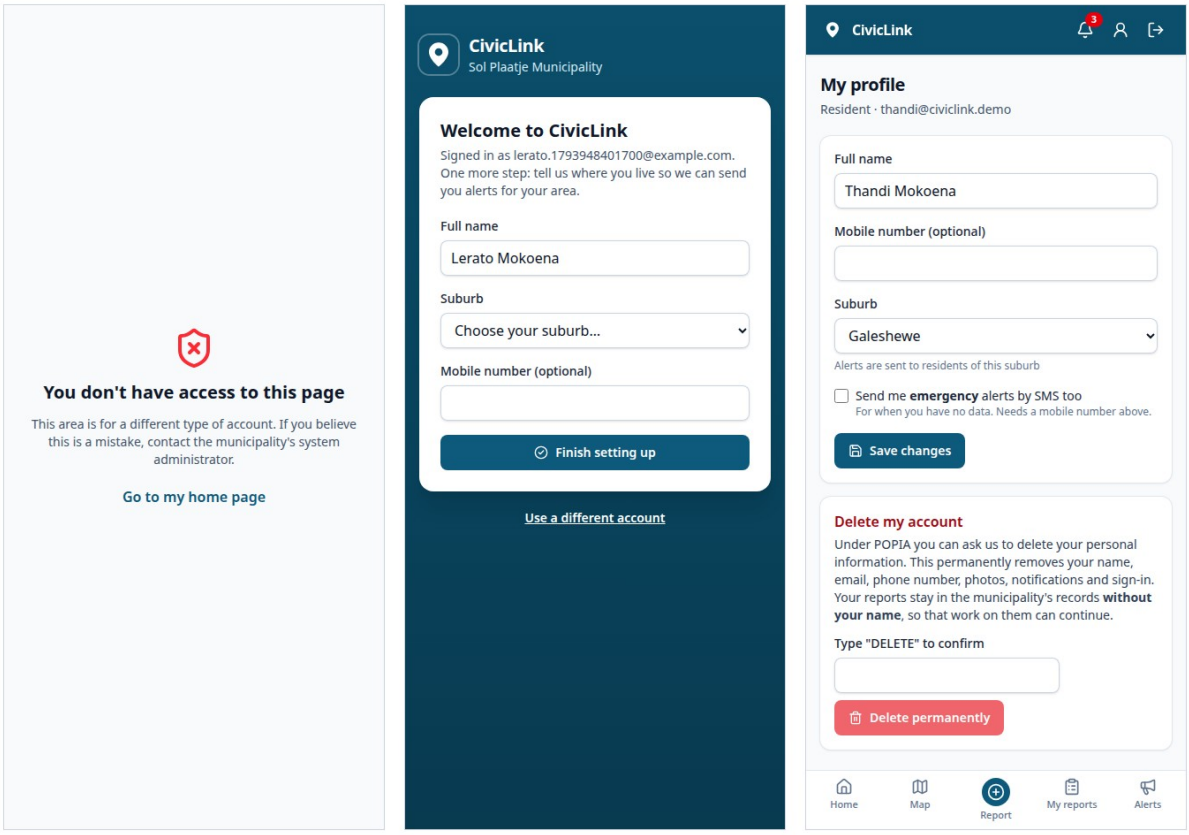


Figure 24: Administrator, SMS outbox



(a) Access denied (b) Google: first sign-in (c) Profile and deletion

Figure 25: Access and accounts: permission denied, Google first sign-in, POPIA deletion

5.6 Data Design

All persistent data lives in **Cloud Firestore**, a document database with real-time listeners and transactions. I also store photos in Firestore, as binary documents in the photos collection (decision D2), so there is no separate file store. Figure 26 shows the document model: each collection is drawn as an entity, and because Firestore has no foreign keys, the references between documents are drawn as relationships with their cardinalities.

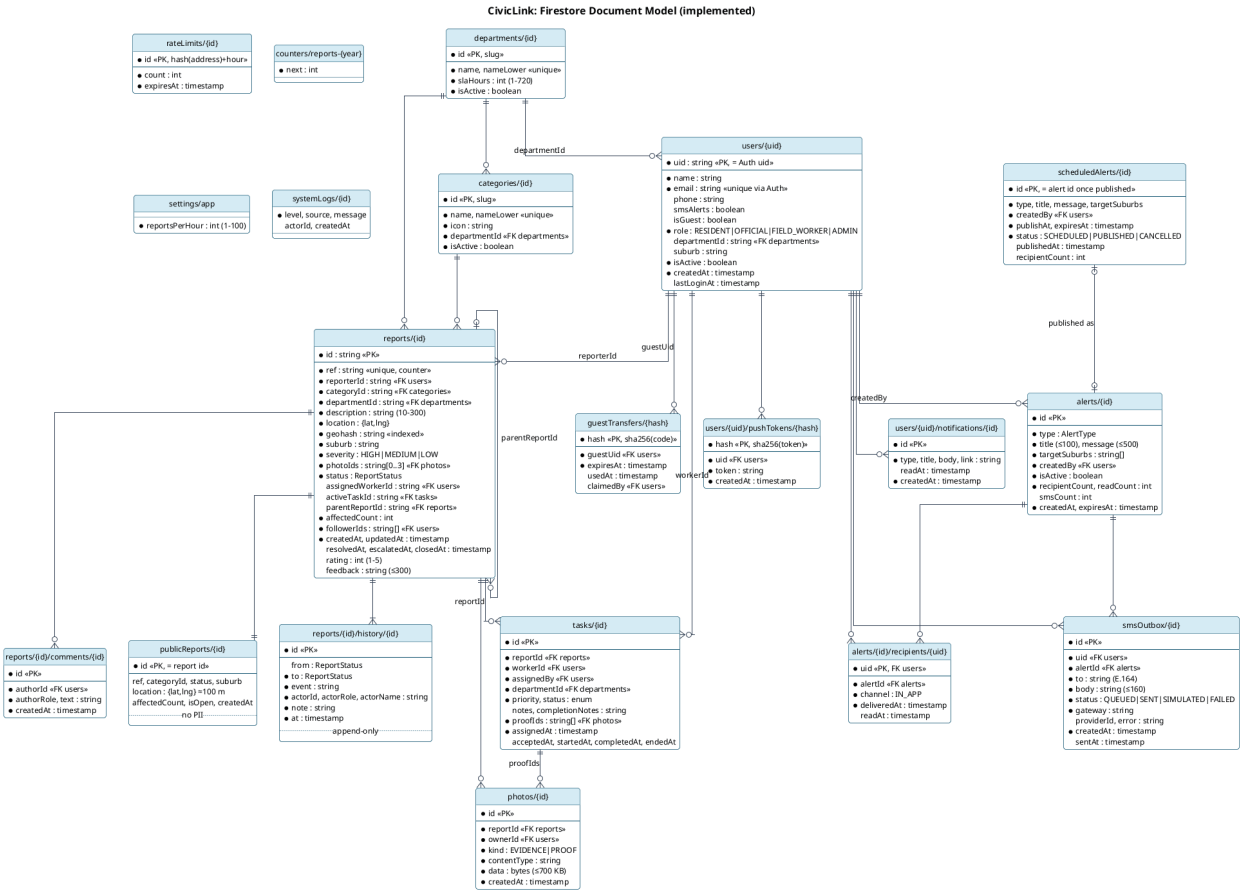


Figure 26: Firestore document model

Table 11: Data dictionary (principal collections)

Collection / field	Type	Key / required	Purpose
users/{uid}		uid = Auth user id (PK)	Profile for every account
name, email	string	required	Display name; sign-in email (unique in Firebase Auth)
role	enum RESIDENT, OFFICIAL, FIELD_WORKER, ADMIN	required	Authorisation, mirrored as a custom claim
departmentId	string	FK departments; required for officials and workers	Department scope (BR-20)
suburb	string	required for residents	Alert targeting (BR-14)
phone, smsAlerts	string, boolean	optional	Mobile number; SMS opt-in for emergency alerts (FR-25)
isGuest	boolean	optional	Guest session without an account (FR-39); also a guest

Collection / field	Type	Key / required	Purpose
			claim
isActive	boolean	required	Deactivation (BR-16)
reports/{id}		auto id (PK)	Incident report
ref	string	required, unique	CL-YYYY-NNNN (BR-04)
reporterId, reporterName	string	FK users, required	Reporter (staff-only visibility)
categoryId, departmentId	string	FK, required	Routing
description	string (10–300)	required	Resident description
location, geohash	{lat,lng}, string	required	Map position; geohash for the 100 m duplicate query
suburb, landmark	string	suburb required	Human-readable location
severity	enum HIGH, MEDIUM, LOW	required	Priority of the resulting task
photoIds	string[0..3]	required (may be empty)	Ids of evidence photos (photos)
status	enum (8 states)	required	Lifecycle (Figure 7)
assignedWorkerId, activeTaskId	string	FK, optional	Current assignment (BR-06)
parentReportId, followerIds, affectedCount	string, string[], int	optional / required	Duplicate linkage (BR-03)
resolutionNotes, resolutionPhotoPaths	string, string[]	optional	Copied from the completed task
createdAt, updatedAt, resolvedAt, escalatedAt, closedAt	timestamp	createdAt/updatedAt required	Lifecycle times, SLA and analytics
rating, feedback	int 1–5, string (≤ 300)	optional	Resident's rating when confirming (FR-37)
reports/{id}/history/{id}		auto id	Append-only audit trail: from, to, event, actorId, actorRole, actorName, note, at
reports/{id}/comments/{id}		auto id	Internal notes: authorId, authorRole, text, createdAt
publicReports/{id}		id = report id	Privacy-safe projection: ref, category, status, location rounded to about 100 m, suburb, isOpen
tasks/{id}		auto id	Assignment: reportId (FK), workerId (FK), assignedBy, departmentId, priority, status,

Collection / field	Type	Key / required	Purpose
			notes, completionNotes, proofPaths, timestamps
alerts/{id}		auto id (or the scheduled alert's id)	type, title (≤ 100), message (≤ 500), targetSuburbs[], createdBy (FK), isActive, recipientCount, readCount, smsCount, createdAt, expiresAt
scheduledAlerts/{id}		auto id	Alert content plus publishAt, status SCHEDULED/PUBLISHED/CANCELLED, publishedAt, recipientCount (FR-28; staff-only)
smsOutbox/{id}		auto id	uid (FK), to (E.164), body (≤ 160), alertId (FK), status QUEUED/SENT/SIMULATED/FAILED, gateway, providerId, error, createdAt, sentAt (FR-25; administrators only)
alerts/{id}/recipients/{uid}		uid (PK, FK users)	Delivery record: channel, deliveredAt, readAt
users/{uid}/notifications/{id}		auto id	type, title, body, link, readAt, createdAt
departments/{id}, **categories/{id}**		slug id	name (unique), slaHours (1–720) / icon and departmentId (FK), isActive
users/{uid}/pushTokens/{hash}		SHA-256 of token	uid, token, createdAt (FR-24; server-only)
photos/{id}		auto id	reportId (FK), ownerId (FK users), kind EVIDENCE/PROOF, contentType, size, data (bytes ≤ 700 KB), createdAt; readable by whoever can read the report
guestTransfers/{sha256(code)}		hash of the one-time code	guestUid (FK users), expiresAt, usedAt, claimedBy (FR-39; server-only)
rateLimits/{id}		address hash + hour	count, expiresAt: guest

Collection / field	Type	Key / required	Purpose
			sessions per network address (BR-26; server-only; the address itself is not stored)
counters/reports-{year}		year	next reference sequence
settings/app		fixed id	reportsPerHour (BR-21)
systemLogs/{id}		auto id	level, source, message, actorId, createdAt

Integrity mechanisms. Firestore does not enforce a schema, so I keep the data consistent in five ways:

1. **Server-only writes.** Clients cannot write reports, tasks, history, alerts or configuration at all; they may only stamp their own last-login time and mark their own notifications read.
2. **Shared schema validation.** Every command validates its input with the same Zod schemas the forms use.
3. **Transactions.** TX-1 (submission: reference counter, duplicate search, report, history, projection, notifications), TX-2 (assignment: one-active-task check, eligible worker, task, report, history) and TX-3 (completion: task and report resolved together) each commit atomically. Firestore retries a transaction on contention, so of two concurrent assignments only one can succeed (test IT-ASG-07).
4. **Append-only history.** History entries are never updated or deleted.
5. **Stable references.** Departments and categories are deactivated rather than deleted (BR-19), so existing references stay valid.

Data-organisation decisions:

- History, comments, recipients and notifications are **subcollections**, so their access rules follow the parent document.
- **Duplicated fields** (names, category names) avoid extra reads in lists, accepting that a renamed user keeps their old name on past records.
- A **geohash field** supports the 100 m proximity query, and **composite indexes** are declared in `firestore.indexes.json`.

5.7 Security, Privacy and Access Control

Authentication. Users sign in with an email and password or with Google, through Firebase Authentication, and guests sign in anonymously (FR-39). CivicLink never stores passwords

itself. When an account is created or its role changes, the server sets signed custom claims (`role`, `dept`, `suburb`, `active`, and `guest` for guests) that the Security Rules use. Every command also reloads the caller's profile and refuses deactivated accounts, and deactivating someone also disables their sign-in and revokes their sessions. The app refreshes its token when an administrator changes the user's role.

Table 12: Role / permission matrix (✓ allowed; scope in brackets)

Function	Resident	Official	Field Worker	Administrator
Register, sign in (password or Google), reset password, edit own profile	✓	✓ (no registration)	✓ (no registration)	✓ (no registration)
Delete own account (POPIA)	✓	—	—	—
"Me too" on an open issue	✓	—	—	—
Turn push notifications on/off for own device	✓	✓	✓	✓
Submit report (online or saved offline); view own reports; confirm (with rating)/reopen	✓	—	—	—
Opt in to SMS for emergency alerts	✓	—	—	—
View community map	✓	✓	—	✓
View report queue and full report details	—	✓ (own dept)	✓ (assigned)	✓
Triage, reject, assign (single or bulk), reassign, escalate	—	✓ (own dept)	—	✓
Accept, start, complete task	—	—	✓ (own tasks)	—
Internal comments	—	✓ (own dept)	✓ (assigned)	✓
Broadcast, schedule, cancel and end alerts; alert history	—	✓	—	✓
View alerts feed	✓ (own suburb)	✓	—	✓
Analytics	—	✓	—	✓
Manage users, departments, categories; system log; SMS outbox	—	—	—	✓

The public home and transparency pages need no account. A **guest** (FR-39) can do what a Resident can, except opt in to SMS or edit a profile, and has the lower limits in BR-26.

Table 13: Security controls

Area	Risk	Implemented control
Authentication	Unauthorised access	Firebase Authentication with email/password (minimum 8 characters) or Google; signed ID tokens on every request; new Google users can only become Residents
Account enumeration	Attackers discover which emails are registered	Password reset and sign-in errors give the same message whether or not an account exists
Abuse	Flooding the queue with fake reports	Configurable per-resident hourly limit checked before the submission transaction (BR-21); "Me too" counted once per resident (BR-23)
Push tokens	Notifications sent to the wrong person or a stale device	Tokens stored server-side per user (no client access), sent only after the command commits, pruned when FCM reports them dead, deleted with the account
Authorisation	A user reaches another role's data or functions	Firestore Security Rules per the matrix (photos inherit their report's access); role and department checks in every command; client route guards for usability only
Account lifecycle	A deactivated user keeps access	Auth account disabled, refresh tokens revoked, profile check on every command
Input	Malicious or invalid data	Zod schemas on the server; geographic bounds; length limits; photo size limit and real-type check from magic bytes, so renamed or script-bearing files (e.g. SVG) are refused; request body limited to 4 MB
Integrity	Race conditions or partial updates	Firestore transactions TX-1 to TX-3; append-only history; no client writes
File uploads	Malicious or oversized files	Photos accepted only through commands, re-checked server-side (type sniffing, ≤ 700 KB); never served as HTML; displayed from bytes with an explicit image content type
Sensitive data (POPIA)	Exposure or indefinite retention of residents' identity, location or photos, contrary to the Protection of Personal Information Act (2013)	Reports readable only by the reporter, followers and staff; public map uses an anonymised projection with location rounded to about 100 m; minimal data collected; processing region africa-south1; residents can erase their personal data (FR-32), leaving anonymised service records (BR-22)
Secrets	API key or credential exposure	Service-account key and cron secret only in Vercel's encrypted environment; none in the repository (`.env.example` lists names only); gitleaks scan in CI; the cron endpoint compares the secret in constant time
Error handling	Leaking internals	Unexpected errors return a generic message and are written to the system log

Area	Risk	Implemented control
Browser attacks	Script injection, clickjacking, protocol downgrade	Enforced Content Security Policy (scripts only from the site and Google sign-in, no eval, no inline script, no framing), HSTS, X-Frame-Options DENY, nosniff (NFR-14)
Public endpoint	Personal data leaking to anonymous users; load on the database	Only aggregates are returned (no names, text, photos or coordinates, test IT-PUB-01); responses come from the CDN cache
Guest abuse	Floods of reports from throwaway sessions	Guests are real authenticated sessions; 3 reports per hour per guest and 10 guest sessions per hour per network address (hashed); transfer codes are random 256-bit, stored only as a hash, single-use and valid 15 minutes (BR-26)
Repeated submissions	A report saved offline being stored twice	Idempotent submission with a device-chosen client id, scoped to the user (D3)
SMS	Phone numbers exposed; messages to people who did not ask	Opt-in with a validated number; SMS records readable by administrators only, numbers masked on screen and erased with the account (BR-25)
Logging and audit	Unaccountable actions	Immutable status history with actor; administrator actions and errors in the system log

Data classification.

Data	Class	Visible to
Names, emails, phone numbers	Personal	Owner, staff
Report photos and exact locations	Personal (may identify a home)	Reporter, staff
Report category, status and approximate location	Public	All signed-in users
Aggregate figures (counts, averages, ratings)	Public	Anyone (transparency page)
Passwords	Secret	Firebase Auth only

5.8 Deployment and Runtime Design

The same code runs in two configurations (Figure 27):

- **Deployed (free tier):** the static web app and the Node command API run on **Vercel** (serverless functions in the Cape Town region). Vercel's CDN serves the hashed assets with a one-year immutable cache and the public figures from its edge cache. Authentication and Firestore run on Firebase's free **Spark** plan, and push goes through

Firebase Cloud Messaging. The API uses a service-account key held in Vercel's encrypted environment. A GitHub Actions schedule calls the protected maintenance endpoint hourly, and Vercel Cron calls it daily. The deployed URL is given in Appendix 11.2.

- **Local:** the web app runs on Vite. Auth, Firestore and Cloud Functions run in the Firebase Emulator Suite on one machine, with the same command registry served by the Cloud Functions adapter. This configuration is used for development, all automated tests and an offline demonstration.

In both, the browser loads map tiles from OpenStreetMap over HTTPS, and the service worker keeps up to 400 recently viewed tiles so maps still show offline.

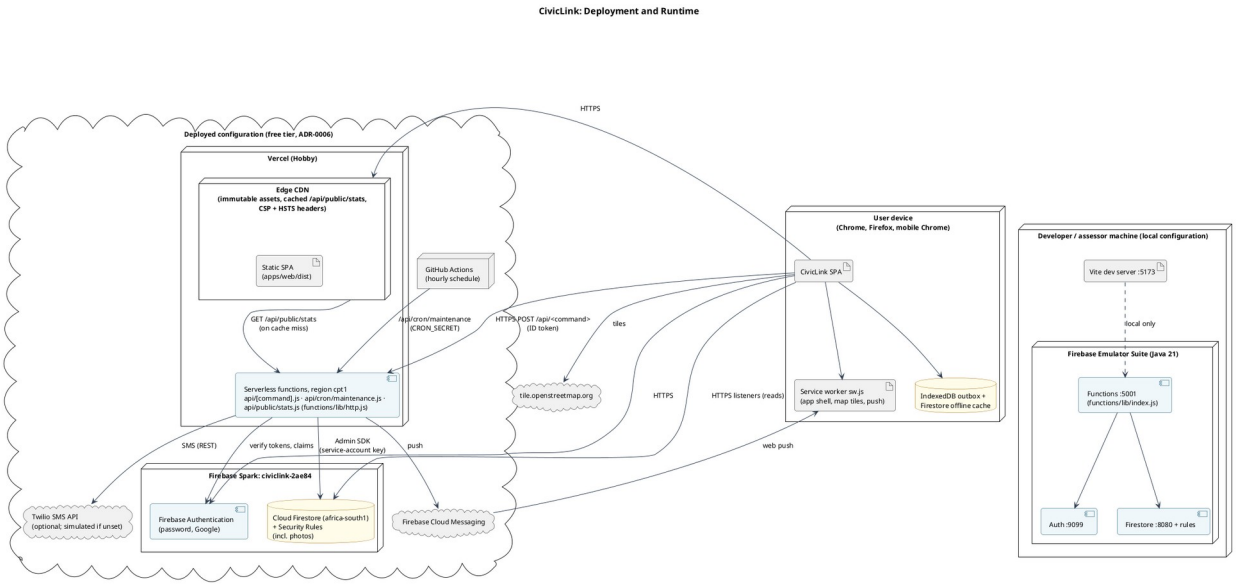


Figure 27: Deployment diagram

6. SYSTEM IMPLEMENTATION

6.1 Codebase and Repository Structure

The repository is a monorepo using npm workspaces, and each top-level folder matches a component of the architecture in Figure 9.

Table 14: Repository structure

Path	Architecture component	Contents
apps/web/	Web application	src/pages/ {auth,resident,official,worker,admin,common,public}, src/components (UI kit, maps, timeline,

Path	Architecture component	Contents
		notification bell, PWA, outbox), src/auth (AuthProvider, RequireRole), src/lib (Firebase setup, typed API client, live hooks, uploads, push, offline outbox), public/sw.js (service worker)
functions/	Command API and schedules	src/index.ts (callable and scheduled exports), src/lib/command.ts (auth/role/validation wrapper), src/services/* (business services), test/ (integration tests)
packages/shared/	Shared domain package	src/schemas.ts, stateMachines.ts, policies.ts, analytics.ts, models.ts, constants.ts; test/ (unit tests)
firestore.rules, firestore.indexes.json	Access control and indexes	Security Rules and composite indexes
api/, vercel.json, functions/src/http.ts, functions/src/commands.ts	Deployed API and hosting	Vercel entry points (commands, cron, public figures); Node HTTP adapter; the single command registry shared with Cloud Functions; cache and security headers
scripts/seed.ts	Data initialisation	Deterministic demonstration data
tests/rules, tests/e2e, tests/pwa, tests/support	Verification	Rules tests, Playwright end-to-end and PWA tests, emulator harness
docs/	Documentation	Baseline SRS, change log, ADRs, diagrams (source and rendered), traceability matrix, test catalogue, defect log, evidence, report

The principal dependency and build files are the root package.json (workspaces, scripts), the per-workspace package.json files, package-lock.json, firebase.json (emulators, hosting, functions), tsconfig.base.json, vitest.config.ts, playwright.config.ts, eslint.config.js and .github/workflows/ci.yml. Cloud Functions are bundled with esbuild, so the shared package is included in the deployable artefact.

6.2 Implementation of Core Capabilities

Table 15: Capability implementation

Capability	Requirement(s)	Main implementation components
Register and sign in	FR-01, FR-02	RegisterPage, LoginPage, AuthProvider, RequireRole; registerResident → UserService; command()/loadActor;

Capability	Requirement(s)	Main implementation components
		users, Auth claims
Submit report with duplicate detection	FR-03–FR-05	NewReportPage, LocationPicker, upload.ts; submitReport → ReportService.submit (TX-1), DuplicatePolicy, formatReference; reports, history, publicReports, counters, photos (photoService: decode, size, type sniffing)
Track reports	FR-06, FR-15	MyReportsPage, ResidentReportPage, Timeline, NotificationBell, useLiveQuery/useLiveDoc listeners; reports, history, notifications
Triage and dispatch	FR-08–FR-10, FR-12, FR-13	ReportQueuePage, OfficialReportPage; ReportService.triage/reject/assign (TX-2)/reassign/escalate; applyTransition; tasks, reports, history
Field completion	FR-11	WorkerHome, TaskPage; updateTask, completeTask → TaskService (TX-3), TaskStateMachine; tasks, reports, photos
Confirm, reopen, SLA	FR-13, FR-14	ResidentReportPage; ReportService.confirm/reopen; MaintenanceService with escalateOverdue and autoCloseResolved schedules and admin runMaintenance
Alerts	FR-16–FR-18	NewAlertPage, AlertsAdminPage, AlertsFeedPage, useResidentAlerts; AlertService; alerts, recipients
Push notifications	FR-24	ProfilePage (PushCard), lib/push.ts, public/sw.js; register/unregisterPushToken; pushService (intents recorded by NotificationService, delivered by execute() after commit)
Me too and offline	FR-33, FR-34, FR-27	CommunityMapPage (MeToo) → affirmReport → ReportService.affirm; public/sw.js, manifest, components/pwa.tsx; lib/outbox.ts (IndexedDB) and components/outbox.tsx (OutboxSync, OutboxCard) → submitReport with clientId
Scheduled alerts and SMS	FR-28, FR-25	NewAlertPage ("Schedule for later"), AlertsAdminPage (Scheduled tab) → broadcastAlert / cancelScheduledAlert → AlertService (scheduledAlerts, publishDue); MaintenanceService.runAll; smsService (SmsGateway: Twilio or simulated) → smsOutbox; ProfilePage opt-in; LogsPage SMS outbox
Bulk assignment	FR-35	ReportQueuePage (checkboxes, BulkAssignBar) → assignReports → ReportService.assignMany (one TX-2 per report)
Public home and guests	FR-38, FR-39	PublicHomePage → GET /api/public/feed →

Capability	Requirement(s)	Main implementation components
		PublicFeedService (publicReports, active alerts); /report (ReportEntry) → AuthProvider.startGuest (anonymous sign-in) → startGuestSession → GuestService.start; GuestAccountPage → linkWithCredential/linkWithPopup + upgradeGuest, or startGuestTransfer + sign-in + claimGuestReports
Transparency and rating	FR-36, FR-37	TransparencyPage → GET /api/public/stats → PublicStatsService (computeAnalytics, openBySuburb, summariseRatings); ResidentReportPage stars → confirmResolution with rating
Accounts	FR-01, FR-02, FR-29 –FR-32	LoginPage, RegisterPage, WelcomePage, ProfilePage, AuthProvider (Google popup, password reset); registerResident, completeProfile, updateProfile, deleteMyAccount → UserService; users, Auth claims, photos
Insight and administration	FR-19–FR-23, FR-26	AnalyticsPage → AnalyticsService → computeAnalytics; UsersPage → UserService; ConfigPage → ConfigService; LogsPage, logService; CommentService; toCsv

Validation. The forms use the same Zod schemas as the server, so the user sees field messages as they type and they disappear once fixed. The server validates every request again anyway, and returns invalid-argument with a fields map that the app shows next to the right inputs.

Business rules. Every status change goes through one function, applyTransition. It asks ReportStateMachine.next(from, event, actor) whether the change is allowed (it throws if not), then updates the report, adds a history entry, updates the public map entry and creates the notifications, all inside the caller's transaction. Rules that depend on data, such as one active task per report (BR-06), worker eligibility (BR-07) and the duplicate search (BR-03), are checked again inside the transaction, so if two people act at the same moment Firestore retries one of them instead of letting the rule break.

Duplicate detection (BR-03). Each report stores a geohash of its location. When a new report comes in, the transaction queries the geohash ranges that cover a 100 m circle (with geofire-common), then DuplicatePolicy filters the results: same category, still open, at most 72 hours old and at most 100 m away by exact distance. The closest match becomes the original.

Error handling. Expected problems are thrown as typed DomainErrors (or state-machine errors) and turned into callable error codes such as invalid-argument, failed-precondition and permission-

denied. Anything unexpected is logged in the system log and the user only sees a general message. The app turns network and timeout errors into friendly messages, and if a screen cannot load (for example offline) it shows a retry screen instead of a blank page.

6.3 APIs, Integrations and External Dependencies

Command API. The command API is a set of HTTPS callable functions that the app calls with the user's ID token. There are no API calls for reading; the app reads Firestore directly, under the Security Rules.

Table 16: Callable command API

Command	Roles	Purpose	Notable errors
registerResident	public	Create resident account	already-exists, invalid-argument
completeProfile	signed in, no profile yet	Finish first Google sign-in as a Resident	already-exists
updateProfile	any role	Own name, phone; residents' suburb	invalid-argument, permission-denied
deleteMyAccount	Resident	POPIA erasure with anonymisation	invalid-argument (confirmation)
affirmReport	Resident	"Me too" on an open issue	failed-precondition (own/closed)
registerPushToken, unregisterPushToken	any role	Device push registration	invalid-argument
submitReport	Resident	TX-1 submission	invalid-argument, permission-denied
triageReport, rejectReport, escalateReport	Official, Admin	Lifecycle actions	failed-precondition, permission-denied
assignReport, reassignTask	Official, Admin	TX-2 dispatch	failed-precondition, invalid-argument
updateTask, completeTask	Field Worker	Task steps, TX-3 completion	failed-precondition, permission-denied
confirmResolution, reopenReport	Resident	Respond to resolution (optional 1–5 rating)	failed-precondition, invalid-argument
addComment	Staff	Internal note	permission-denied
broadcastAlert, deactivateAlert	Official, Admin	Alerts, sent now or scheduled (publishAt)	invalid-argument
startGuestSession	signed in (anonymous)	Create the guest profile (FR-39)	resource-exhausted (per-address limit)
upgradeGuest, startGuestTransfer, claimGuestReports	Resident (guest / account)	Keep guest reports: upgrade in place, or move into an existing	invalid-argument, failed-precondition, not-found

Command	Roles	Purpose	Notable errors
		account	
sendTestSms	Admin	Check the SMS gateway	invalid-argument
cancelScheduledAlert	Official, Admin	Cancel a scheduled alert	failed-precondition (already published)
assignReports	Official, Admin	Bulk TX-2 dispatch (1–20 reports)	invalid-argument; per-report failures listed
markAlertRead	Resident	Read receipt	not-found
createStaffUser, setUserRole, setUserActive	Admin	Accounts	permission-denied (self)
upsertDepartment, upsertCategory, updateSettings	Admin	Configuration, report limit	invalid-argument
getAnalytics	Official, Admin	Metrics	—
runMaintenance	Admin	Run the scheduled rules on demand (SLA, auto-close, scheduled alerts)	—

Besides the commands, the deployed API has three plain HTTP routes: `GET /api/public/stats` (anonymous figures for FR-36), `GET /api/public/feed` (open issues and active alerts for FR-38), both without sign-in and cacheable by the CDN, and `GET /api/cron/maintenance` (the scheduled rules, protected by a secret).

External integrations.

- **OpenStreetMap tiles** are fetched by the browser over HTTPS. If tiles are unavailable, the map shows a blank background but still works: the pin, map clicks and coordinates function, and the report form, lists and timelines do not depend on tiles.
- **Browser geolocation** is optional. If permission is denied, or the position is outside the municipality, the user is told to drag the pin instead.
- **Firebase Cloud Messaging** delivers web push. Messages are sent only after a command commits; if FCM is unavailable, in-app notifications still arrive, and tokens FCM reports as dead are removed.
- **SMS gateway (Twilio)** is behind an `SmsGateway` interface. With `TWILIO_ACCOUNT_SID`, `TWILIO_AUTH_TOKEN` and `TWILIO_FROM` set, messages are sent through Twilio's REST API with an 8-second timeout; without them a simulated gateway records each message as Simulated. A failure marks that message Failed and is logged, and never stops the alert (IT-SMS-02).

- **Vercel CDN** caches the hashed assets and the public figures (NFR-13).

Two adapters, one API. All 36 commands are defined once in functions/src/commands.ts. index.ts serves them as Cloud Functions (used with the local emulator) and http.ts serves them as a plain Node handler on Vercel. The Node version verifies the Firebase ID token (and rejects revoked sessions), limits the request size and returns errors in the same callable format, so the app only has to switch to httpsCallableFromURL. Locally, Firebase itself comes from the official Emulator Suite; nothing is mocked in the application code.

6.4 AI / Machine-Learning Component

Not applicable. CivicLink has no AI or machine-learning component. Duplicate detection, SLA escalation, the analytics and the public figures are all fixed rules, specified in Section 4.3 and unit-tested.

7. VERIFICATION AND TESTING

7.1 Test Approach

I tested the system at four automated levels plus a set of non-functional checks, and all of them run on every push in GitHub Actions (.github/workflows/ci.yml). Every test has its ID in its name, so any row in this section can be found in the code (for example grep -r "IT-ASG-07").

Table 17: Test levels

Level	Tool	Scope	Count
Unit	Vitest	Shared domain: schemas, state machines, duplicate/SLA/resolution policies, location helpers, image sniffing, references, analytics, ratings, SMS text, schedule rules, CSV (≈ 98% line coverage)	41
Security rules	@firebase/rules-unit-testing on the emulator	Allow and deny case for every permission-matrix cell, including photos, scheduled alerts and SMS records	22
API integration	Vitest + Firebase client SDK against the emulators	Every command: success, validation, boundary, authorisation, state conflict and concurrency; the Node HTTP adapter and public endpoint; web push and SMS with fake senders	99
Installable PWA	Playwright against the production build (vite preview)	Manifest, icons, service worker control, offline reload (NFT-10)	1
End-to-end	Playwright (Chromium and	Full lifecycle across three browser	20 (run

Level	Tool	Scope	Count
	Firefox; 360 px mobile)	sessions, duplicates, alerts (immediate and scheduled), bulk assignment, offline drafts, transparency page, access control, validation, out-of-area GPS, Google sign-in, password reset, profile and account deletion; timing, mobile and accessibility checks	through both API adapters)
Deployed site	Playwright against the live URL	CDN caching and security headers (NFT-11, NFT-12)	2
Non-functional	Playwright timing, axe-core, gitleaks, ESLint, TypeScript	NFR-04 to NFR-09, NFR-12 to NFR-14	—

Apart from checking how screens look, everything was tested automatically. My **manual testing** was exploratory sessions in each role, checking every screenshot in Section 5.5 (including phone layouts), and a few live checks that cannot be automated, such as a push notification and an SMS arriving. The seed script (`scripts/seed.ts`) gives every test run the same starting data.

7.2 Functional and Business-Rule Verification

The table below shows the main test cases. The full list of 185 automated tests, with their results for the submitted commit, is in Appendix 11.3. All of them passed; "As expected" means the actual result matched the expected result exactly.

Table 18: Functional and business-rule test results

Test ID	Requirement / rule	Test scenario	Type	Expected result	Actual result	Status
E2E-01	SC-01, FR-03 –FR-14	Resident submits → official triages and assigns → worker accepts, starts, completes with proof → resident sees Resolved and confirms	End-to-end	Report Closed; full timeline; updates live	As expected	Pass
IT-RPT-01	FR-03, FR-04	Valid submission with photo	Integration	Submitted, CL- reference	As expected	Pass
IT-RPT-02	BR-02	Short description and unknown suburb	Validation	invalid-argument with both field errors; nothing written	As expected	Pass
IT-RPT-03	BR-02	Location in Johannesburg	Boundary	Refused	As expected	Pass

Test ID	Requirement / rule	Test scenario	Type	Expected result	Actual result	Status
IT-RPT-04	BR-02	Photo path of another user / missing file	Authorisation	Refused	As expected	Pass
IT-RPT-06	BR-01	Official, worker and anonymous submit	Authorisation	permission-denied / unauthenticated	As expected	Pass
IT-RPT-08	BR-04, NFR-10	12 concurrent submissions	Concurrency	12 unique, consecutive references	As expected	Pass
UT-DUP-01..04	BR-03	99/100/101 m; 71/72/73 h; other category; closed original	Boundary	Duplicate only within 100 m, 72 h, same category, open	As expected	Pass
IT-RPT-09, E2E-02	FR-05	Second resident reports the same issue 33–60 m away	Integration / E2E	Linked as duplicate; original count 2; follower added	As expected	Pass
UT-RSM-01..03	BR-05	Every allowed and every disallowed (status, event, actor)	Unit	Only the transitions of Figure 7 succeed	As expected	Pass
IT-TRI-02	BR-05	Triage an already-reviewed report	State conflict	failed-precondition	As expected	Pass
IT-TRI-03	BR-12	Reject with a 2-character reason	Validation	Refused; valid reason → Rejected	As expected	Pass
IT-TRI-04	BR-20	Roads official triages a water report	Authorisation	permission-denied	As expected	Pass
IT-ASG-02, -03	BR-07	Assign to a roads worker / deactivated worker	Validation	Refused	As expected	Pass
IT-ASG-04	BR-06	Assign a report with an active task	State conflict	failed-precondition	As expected	Pass
IT-ASG-07	NFR-10	Two officials assign the same report simultaneously	Concurrency	Exactly one succeeds; one task exists	As expected	Pass
IT-ASG-08	FR-12	Reassign	Integration	Old task Reassigned, new task Assigned	As expected	Pass
IT-TSK-02	BR-08	Another worker accepts the task	Authorisation	permission-denied	As expected	Pass
IT-TSK-03	BR-08	Start before accept; complete before start	State conflict	failed-precondition	As expected	Pass
IT-TSK-04	BR-09	Complete with 2-	Validation	invalid-argument	As	Pass

Test ID	Requirement / rule	Test scenario	Type	Expected result	Actual result	Status
		character notes and no photo			expected	
IT-TSK-06	BR-09, NFR-10	Complete a task reassigned while on site	Failure	Refused; report unchanged	As expected	Pass
IT-ESC-01	BR-10	Run schedule with clock 49 h after creation (SLA 48 h); run again	Boundary	Escalated once; second run no change	As expected	Pass
IT-RES-02	FR-14	Reporter reopens with a reason	Integration	Under Review; officials notified	As expected	Pass
IT-RES-03	BR-11	Respond 8 days after resolution; run schedule	Boundary	Refused; report auto-closed	As expected	Pass
IT-RES-04	BR-11	Another resident confirms	Authorisation	permission-denied	As expected	Pass
IT-ALR-01	FR-16, BR-14	Broadcast to Galeshewe	Integration	Delivery record only for the active Galeshewe resident	As expected	Pass
IT-ALR-03	BR-13	Title 101 chars; no suburbs; past expiry; expiry 31 days	Boundary	All refused	As expected	Pass
E2E-03	FR-17, BR-14	Galeshewe and Roodepan residents open alerts	End-to-end	Visible to Galeshewe only	As expected	Pass
IT-AUTH-02	FR-01	Duplicate email; 5-character password	Validation	already-exists / invalid-argument	As expected	Pass
IT-AUTH-03	BR-16	Admin deactivates a signed-in resident who then submits / signs in	Authorisation	Command refused; sign-in refused	As expected	Pass
IT-ADM-04	BR-15	Admin demotes or deactivates own account	Authorisation	permission-denied	As expected	Pass
IT-CFG-01	FR-21	SLA hours 0 / 721 / 96	Boundary	0 and 721 refused; 96 saved	As expected	Pass
IT-CFG-03	BR-19	Deactivate category, then submit with it	Integration	Refused	As expected	Pass
IT-ANL-01, UT-ANL-01	FR-19, SC-05	Metrics on fixture and after a new	Integration	Counts, averages and SLA % exact; no NaN on	As expected	Pass

Test ID	Requirement / rule	Test scenario	Type	Expected result	Actual result	Status
		report		empty data		
E2E-04	NFR-01	Each role opens other roles' pages	End-to-end	Access-denied page; anonymous → login	As expected	Pass
E2E-05	BR-02	Submit report with short text, no location and no severity	Validation	Three field messages; cannot proceed	As expected	Pass
E2E-07	FR-03, BR-02	Device GPS reports Johannesburg	Boundary	Distance explained; no location set; tapping the map sets the pin and suburb	As expected	Pass
IT-RPT-11	BR-21	Limit set to 2; resident submits 3 reports	Boundary	Third refused (resource-exhausted)	As expected	Pass
IT-AUTH-04, E2E-08	FR-29, BR-16	First Google sign-in; client sends role ADMIN	Authorisation	Profile step required; account is a Resident; second completion refused	As expected	Pass
E2E-09	FR-30	Reset for a registered and an unregistered email	Security	Identical confirmation for both	As expected	Pass
IT-DEL-01, E2E-10	FR-32, BR-22	Resident deletes account (wrong confirmation, then DELETE)	Integration	Refused, then reports anonymised; photos, profile, receipts and Auth account removed; sign-in fails	As expected	Pass
IT-AFF-01, E2E-11	FR-33, BR-23	Second resident taps "Me too" on the map, then again	Integration / E2E	Count 2, follower added, one timeline note; repeat says already confirmed	As expected	Pass
IT-AFF-02	BR-23	"Me too" on own report, by an official, and on a rejected report	Authorisation / state	Refused	As expected	Pass
IT-PSH-02	FR-24	Official triages a report; then repeats the triage (refused)	Integration	Exactly one push to the reporter's device; none for the refused command	As expected	Pass
IT-PSH-03	FR-24	Suburb alert with one live and one dead device token	Failure	Both targeted; dead token pruned	As expected	Pass
IT-DEL-02	BR-22	Staff self-delete; triage an anonymised report	Authorisation	Refused; anonymised report progresses without notifications	As expected	Pass

Test ID	Requirement / rule	Test scenario	Type	Expected result	Actual result	Status
IT-OFF-01, E2E-12	FR-27	Report saved offline, then sent on reconnect; the same report sent twice	Integration / E2E	Saved on the phone; sent automatically with a reference; stored once (second send returns the same reference)	As expected	Pass
IT-OFF-02	FR-27	Another resident reuses the client id; a malformed id	Security / validation	New, separate report; malformed id refused	As expected	Pass
IT-SCH-01	FR-28, BR-24	Schedule an alert; run the schedule before and after its time, then again	Integration / boundary	Not visible before; published once with the same id and delivery records	As expected	Pass
IT-SCH-02, E2E-13	FR-28, BR-24	Publish time in 1 minute; after expiry; cancel, then cancel again; resident cancels	Validation / state / authorisation	Refused; cancelled alert never published; second cancel refused; resident refused	As expected	Pass
IT-BLK-01, E2E-14	FR-35, BR-06	Bulk-assign two triaged reports and one already assigned	Integration / E2E	Two assigned to the worker; the third listed with the reason	As expected	Pass
IT-BLK-02	FR-35, BR-20	Roads official bulk-assigns a water report; resident calls it; 21 reports	Authorisation / boundary	Not assigned; refused; refused	As expected	Pass
IT-RAT-01, E2E-01	FR-37	Rating 6, then 4 stars with a comment	Validation / integration	6 refused; rating and comment stored and shown	As expected	Pass
IT-PUB-01, E2E-15	FR-36, NFR-11	Anonymous visitor opens the transparency page	Integration / E2E / privacy	Figures shown without sign-in; no names, descriptions, comments or coordinates in the response; CDN cache header present	As expected	Pass
IT-SMS-01	FR-25, BR-25	Opt in without a number; Water and Emergency alerts to an opted-in resident	Validation / integration	Opt-in refused without a number; one SMS (+27…, ≤ 160 characters) for the emergency only	As expected	Pass
IT-SMS-02	FR-25	Gateway down during an emergency alert	Failure	Alert delivered; SMS marked Failed with the reason	As expected	Pass
IT-GST-01	FR-39	Anonymous session	Integration /	Guest Resident profile	As	Pass

Test ID	Requirement / rule	Test scenario	Type	Expected result	Actual result	Status
		starts a guest; starts again; no session	authorisation	and claim; second start changes nothing; unauthenticated refused	expected	
IT-GST-02	FR-39, BR-26	Guest submits four reports in an hour; tries to opt in to SMS	Boundary / authorisation	Three accepted, fourth refused with "create an account"; opt-in refused	As expected	Pass
IT-GST-03, E2E-16	FR-39	Guest reports, then creates an account (email/password)	Integration / E2E	Same user id; report kept and shows the new name; upgrade before linking and a second upgrade refused	As expected	Pass
IT-GST-04, E2E-17	FR-39	Guest reports, then signs in to an existing account	Integration / E2E	Report moved; guest deleted; code refused on reuse; unknown code not found	As expected	Pass
IT-GST-05	BR-26	11 guest sessions from one address in an hour	Boundary / security	Eleventh refused (429); another address unaffected	As expected	Pass
IT-PUB-03, E2E-16	FR-38, NFR-11	Visitor opens the public home	Integration / E2E / privacy	Active alerts and open issues shown; only rounded locations; no names or descriptions; 60 s cache header	As expected	Pass
IT-SMS-04, IT-SMS-05	FR-25	Admin test SMS without and with Twilio credentials (Twilio API stubbed); Twilio error	Integration / contract	Simulated; then the documented Twilio request (URL, Basic auth, To/From/Body) and SENT; error recorded as FAILED	As expected	Pass
IT-SMS-03	BR-25, BR-22	Remove the phone number; delete the account	Integration / privacy	Opt-in ends; SMS records erased	As expected	Pass

7.3 Non-Functional and Security Verification

Table 19: Non-functional verification results

NFR	Test	Result	Status
NFR-01 Authorisation	RT-* (22 tests: every matrix cell, e.g. a resident cannot read another's report or list all reports, a roads official cannot read water reports, no client can write	All denied cases denied; all allowed cases allowed	Pass

NFR	Test	Result	Status
	reports/tasks/history) and API authorisation tests		
NFR-02 Authentication	RT-USR-02 (users cannot change their own role, department or active flag); review: no password fields in Firestore	Pass	Pass
NFR-03 Validation	IT-RPT-02, IT-TRI-03, IT-ALR-03, IT-CFG-01, E2E-05	Field-level errors, no data written	Pass
NFR-04 Secrets	gitleaks over the full git history in CI	0 findings	Pass
NFR-05 Propagation	NFT-05 inside E2E-01	Submit → official queue 0.07 s; completion → resident screen 1.90 s (target 5 s)	Pass
NFR-06 Responsiveness	NFT-06	Dashboard 1.20 s, queue 1.32 s, alerts 0.95 s (target 3 s); p95 of 50 commands 73 ms (target 2 s)	Pass
NFR-07 Mobile	NFT-07 at 360×640 on resident and worker screens; report form has 3 steps	No horizontal overflow	Pass
NFR-08 Accessibility	NFT-08 axe-core audit against WCAG 2.1 A/AA (W3C, 2018) on login, report form, my reports, queue	0 violations (after DEF-03)	Pass
NFR-09 Compatibility	E2E suite on Chromium and Firefox in CI	All pass	Pass
NFR-10 Integrity	IT-RPT-08, IT-ASG-07, IT-TSK-06	Atomic, exactly-once outcomes	Pass
NFR-11 Privacy	RT-RPT-03, RT-PUB-01, RT-PHO-01, RT-PHO-02, UT-PRV-01, IT-RPT-07	No PII in public data; photos restricted	Pass
FR-34 Installable / offline	NFT-10 on the production build: manifest and icons, service worker in control, reload with the network off	App shell and offline notice shown	Pass
NFR-12 Maintainability	CI: ESLint, TypeScript strict, coverage threshold	0 lint errors; 97.9% line coverage of the domain package	Pass
NFR-13 Caching and CDN	NFT-11 on the deployed site	Assets `max-age=31536000, immutable`; app shell revalidated; public figures served from the edge cache (HIT on every repeat request)	Pass
NFR-14 Browser hardening	NFT-12 on the deployed site; manual check of every screen and the Google sign-in popup	CSP without `unsafe-eval`, HSTS two years, framing denied, nosniff; 0 CSP	Pass

NFR	Test	Result	Status
		violations	

Timings come from the full local run recorded in Appendix 11.3 (6 November 2026), on the development laptop running the emulators, Vite and the browsers at the same time.

Production verification. The same Playwright tests were run against the deployed system (https://civiclink-five.vercel.app) on 20 October 2026:

Test	Result on the live system
E2E-01 full lifecycle (three browser sessions)	Pass. New report on the official's queue after 0.07 s; resolution on the resident's screen after 1.90 s (NFR-05 target 5 s)
E2E-02 duplicate detection, E2E-07 out-of-area GPS	Pass
E2E-04 access denied per role, E2E-06 wrong password	Pass
NFT-06 page loads	Dashboard 1.02 s, queue 1.06 s, alerts 0.50 s (target 3 s)
NFT-08 accessibility (axe-core)	0 violations
Hourly maintenance schedule (GitHub Actions → /api/cron/maintenance)	Run succeeded with the secret; refused without it
E2E-11 to E2E-17 ("Me too", offline drafts, scheduled alerts, bulk assignment, transparency page, public home with guest reporting and account)	Pass
NFT-08 accessibility, including the public home and transparency pages	0 serious or critical violations
NFT-11 CDN caching, NFT-12 security headers	Pass; public figures served from the edge cache after the first request
SMS gateway (FR-25), administrator test SMS through Twilio	First attempt: authenticated, but refused (recorded as Failed with Twilio's reason) because trial accounts may only send predefined templates (Twilio, 2026b). With TWILIO_TRIAL_TEMPLATE=sms_event_notifications, Twilio accepted the message (status Sent) to a verified South African number. With an upgraded account the real alert text is sent; no code change
Web push (FR-24), manual	An alert broadcast by an official appeared as a system notification in a separate signed-in browser about 5 s later; the test alert was then ended

7.4 Defects and Known Issues

Table 20: Significant defects found and resolved

ID	Found by	Description	Resolution
DEF-01	IT-CFG-02	Creating a department whose name produced an existing identifier silently overwrote that department	Creation now refuses existing identifiers; updates require an existing record
DEF-02	E2E-01	Submissions with an empty optional field were rejected, because the callable protocol sends `undefined` as `null`	The command wrapper treats top-level nulls as absent; regression test IT-RPT-10
DEF-03	NFT-08	Three text styles were below the WCAG AA contrast ratio	Darker text colours
DEF-04	CI on a clean checkout	An over-broad `lib/` ignore rule had kept two source folders out of the repository	Ignore rule narrowed to the build output; CI now builds from a clean clone on every push
DEF-05	E2E-10	The profile form opened with an empty name, because it read the profile before it had loaded	The form is rendered only after the profile loads
DEF-06	IT-RPT-08 in CI	Under concurrency, extra reads inside the submission transaction (rate-limit count, profile, settings, officials) caused lock timeouts	Those reads moved before the transaction; it now contends only on the reference counter
DEF-07	E2E-01 run against production	Submission failed in production because the rate-limit query needed a composite index, which the emulator never requires	Index added and deployed; the E2E suite can now run against the deployed URL
DEF-08	First Vercel deployment	API calls crashed because a firebase-admin dependency pulls in an ESM-only package that Vercel's loader cannot `require()`	Dependency pinned to a compatible version (npm override)
DEF-09	E2E-11 against production	Map markers lost their per-report CSS class in production builds (set through a styling option Leaflet ignores for classes)	Class passed when the marker is created; production E2E runs now use per-run locations because live data persists
DEF-10	Manual push check on the deployed site	Turning push on failed silently, because the web configuration lacked FCM's sender ID	The sender ID is derived from the Firebase app ID; push verified end to end
DEF-11	CSP report-only trial on the deployed site	The validation library probed `eval` on every page, which an enforced CSP would report as a violation	Validation runs in "jitless" mode; the CSP was then enforced with 0 violations
DEF-12	E2E-16	After reporting as a guest, the home screen showed "Welcome" instead of the name:	The listener restarts once claims

ID	Found by	Description	Resolution
		the profile listener started before the guest's claims existed, was refused by the rules and never retried	exist
DEF-13	E2E-16	Starting a guest session from the public home redirected to the resident home instead of the report form	A dedicated /report entry route starts the guest session, then opens the form
DEF-14	Client review of the live map	After the CSP was enforced, map tiles stopped loading for returning visitors: the service worker fetches tiles, and worker fetches fall under connect-src, which did not list the tile servers (first visits, without the worker, were fine)	Tile servers added to connect-src; NFT-12 now checks that tiles load with the worker in control
DEF-15	Firefox console on the live site	Firefox still reported the validation library's eval probe, and returning visitors could keep a service worker installed under an older security policy	The setting moved into the shared package so it applies before any validation; the service worker is now checked for updates on every visit
DEF-16	Client test on a phone (offline)	The installed app could stay on the loading screen offline: the app's code files were saved only as they were used, so a first launch (or a screen never opened online) had nothing to run	The build writes every app file into the service worker, which saves them all at install; a screen that still cannot load shows "not available offline yet" with a retry
DEF-17	Client test on a phone	The notifications panel ran off the left edge of the screen, and long status labels ("Under review") made badges different sizes	On phones the panel spans the screen under the top bar (and opens upwards from the staff sidebar); the label is now "Review" and badges never wrap

There are no open defects. The limitations that affect included features are in Section 3.4.

8. SYSTEM SETUP AND OPERATION

Prerequisites. Node.js 22, Java 21 or later (required by the Firebase emulators) and Git. No Firebase account, API keys or .env file are needed: by default the application connects to the local emulators with a demonstration project ID (demo-civiclink).

Install and run.

```
git clone https://github.com/altechemist/civiclink.git
cd civiclink
```

```
git checkout v1.0.0
npx -y npm@11.6.2 ci
npm run dev
```

`npm run dev` builds the Cloud Functions, starts the Auth, Firestore and Functions emulators, initialises the database with the deterministic seed (departments, categories, 15 accounts, 150 reports with valid histories, and alerts), and starts the web application. Open **http://127.0.0.1:5173**. The emulator data browser is at http://127.0.0.1:4000. Restarting the command resets the database to the same seed. npm 11 is used because npm 10 has a known workspace-resolution defect.

Demo accounts. Locally, all accounts use the password `CivicLink#2026`, and the login page offers one-click shortcuts for the main four. The deployed system at **https://civiclink-five.vercel.app** has the same accounts with a private password, supplied to the lecturer separately. For the demonstration period its login page also offers one-tap demo accounts (enabled by the `VITE_DEMO_PASSWORD` setting, which puts that password in the public app code); the setting is removed after assessment, which also removes the buttons.

Table 21: Demo accounts

Role	Email	Notes
Resident	thandi@civiclink.demo	Galeshewe; reports in every state
Resident	pieter@civiclink.demo	Roodepan
Official	naledi@civiclink.demo	Water & Sanitation (also thabo@ Electricity, johan@ Roads)
Field worker	sipho@civiclink.demo	Water & Sanitation (also bongani@ Electricity, mpho@ Roads)
Administrator	admin@civiclink.demo	All official and administrative functions

Tests. `npm run test:unit`, `npm run test:emulator` (rules and API integration), and `npm run test:e2e` (run `npx playwright install chromium` once beforehand). The results of the latest full run are in Appendix 11.3.

Scheduled rules locally. The emulator does not run schedules by itself. To escalate overdue reports, close old resolved reports and publish due scheduled alerts, sign in as the administrator and select **Run scheduled rules now** on the System log page.

Deployed configuration (free tier). The full runbook is in `docs/deploy.md`. In summary:

6. In the Firebase console, enable Email/Password and Google sign-in and create the Firestore database (location africa-south1).

7. Deploy the rules and indexes with `npx firebase deploy --only firestore --project prod`.
8. Import the GitHub repository into Vercel, which reads the build settings from `vercel.json`.
9. Set the environment variables listed in `.env.example`: Firebase web config, `VITE_API_BASE=/api`, `VITE_USE_EMULATORS=false`, `VITE_FIREBASE_VAPID_KEY` (web push), `FIREBASE_SERVICE_ACCOUNT` and `CRON_SECRET`. Optionally, `TWILIO_ACCOUNT_SID`, `TWILIO_AUTH_TOKEN` and `TWILIO_FROM` switch SMS from simulated to real sending.
10. Set the same `CRON_SECRET` and the `APP_URL` in the GitHub repository for the hourly schedule. Enable the Anonymous sign-in provider for guest reporting (FR-39).
11. Seed the deployed project with `npm run seed:prod`.

Alternative cloud deployment (Blaze plan).

12. Copy `.env.example` to `apps/web/.env.local` and enter the web configuration of a Firebase project on the Blaze plan.
13. Set `VITE_USE_EMULATORS=false` and leave `VITE_API_BASE` empty.
14. Run `npm run build` followed by `npx firebase deploy --project prod`.

The same code also runs entirely on Firebase.

9. SOLUTION EVALUATION

Table 22: Evaluation against the success criteria

Success criterion	Outcome	Evidence
SC-01: The complete incident lifecycle runs end to end	Achieved	E2E-01 passes in Chromium and Firefox across three independent browser sessions (resident, official, field worker); Figures 12–17
SC-02: Key business rules are enforced server-side	Achieved	BR-03 to BR-11 each have unit and integration tests that bypass the user interface and call the API directly (UT-DUP, UT-RSM, IT-ASG-04/07, IT-TSK-03/06, IT-ESC-01, IT-RES-03); all pass
SC-03: Role-based access control holds	Achieved	22 Security Rule tests covering every cell of the permission matrix, plus authorisation cases for every command and E2E-04; all pass
SC-04: Residents receive timely, targeted information	Achieved	NFT-05: report visible to the official in 0.07 s and the resolution visible to the resident in 1.90 s (target 5 s); E2E-03 shows suburb targeting
SC-05: Officials have operational visibility from live data	Achieved	UT-ANL-01 and IT-ANL-01 reproduce the expected metrics; Figure 21

I think the final system does what it set out to do. I built and tested the core lifecycle first, with a smaller scope than the partly simulated Phase 3 prototype. Once that was solid, I added the four capabilities I had deferred at the baseline (SMS, offline drafts, scheduled alerts and bulk assignment) and a few additions (a public home page with guest reporting, a transparency page, service ratings, and caching and browser hardening), each through a change request with its own requirements, rules and tests. Every requirement runs on a real back end, is tested automatically, and the same tests were also run against the deployed system.

If the municipality wanted to use it for real, the next steps would be a paid SMS gateway with a proper sending queue for large broadcasts, a paid scheduler so scheduled alerts never depend on site traffic, and a pilot with one department to check the SLA targets against real workloads.

10. REFERENCES

Auditor-General South Africa. (2024). *Consolidated general report on local government audit outcomes: MFMA 2022-23*. https://www.agsa.co.za/Reporting/MFMAReports/MFMA2022-2023.aspx

Diamond Fields Advertiser. (2026, February 16). *Kimberley residents frustrated as Sol Plaatje Municipality struggles with burst pipes and service delivery* [Opinion]. https://dfa.co.za/diamond-fields-advertiser/opinion/2026-02-16-kimberley-residents-frustrated-as-sol-plaatje-municipality-struggles-with-burst-pipes-and-service-delivery/

Firebase. (2026a). *Authenticate with Firebase anonymously using JavaScript*. Google. https://firebase.google.com/docs/auth/web/anonymous-auth

Firebase. (2026b). *Cloud Firestore documentation*. Google. https://firebase.google.com/docs/firestore

Firebase. (2026c). *Firebase Cloud Messaging: Web push*. Google. https://firebase.google.com/docs/cloud-messaging/js/client

Firebase. (2026d). *Firebase Local Emulator Suite*. Google. https://firebase.google.com/docs/emulator-suite

Firebase. (2026e). *Firebase Security Rules documentation*. Google. https://firebase.google.com/docs/rules

Leaflet. (2023). *Leaflet: An open-source JavaScript library for interactive maps* (Version 1.9.4) [Computer software]. https://leafletjs.com

MDN contributors. (2026). *Content-Security-Policy*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Content-Security-Policy

Meta Platforms. (2026). *React* (Version 19) [Computer software]. https://react.dev

Microsoft. (2026). *Playwright* (Version 1.63) [Computer software]. https://playwright.dev

Noordkaap Bulletin. (2024, January 30). *Sol Plaatje 'has a plan' to solve water crisis*. https://noordkaapbulletin.co.za/sol-plaatje-has-a-plan-20240130/

OpenStreetMap contributors. (2026). *OpenStreetMap* [Data set]. OpenStreetMap Foundation. https://www.openstreetmap.org

Protection of Personal Information Act 4 of 2013. (2013). *Government Gazette, 581*(37067). Republic of South Africa.

Sol Plaatje Local Municipality. (2022). *Integrated development plan 2022–2027*. https://lg.treasury.gov.za/supportingdocs/NC091/NC091_IDP%20Final_2023_Y_20220908T164950Z_kdeklerk.pdf

Statistics South Africa. (2023). *Census 2022: Municipal fact sheet*. https://census.statssa.gov.za/assets/documents/2022/Census_2022_Municipal_factsheet-Web.pdf

Twilio. (2026a). *Programmable Messaging API: Message resource*. https://www.twilio.com/docs/messaging/api/message-resource

Twilio. (2026b). *Twilio trial account*. https://www.twilio.com/docs/usage/trials

Vercel. (2026a). *Caching on the Vercel CDN*. https://vercel.com/docs/edge-cache

Vercel. (2026b). *Vercel Functions documentation*. https://vercel.com/docs/functions

W3C. (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. World Wide Web Consortium. https://www.w3.org/TR/WCAG21/

W3C. (2023). *Web Application Manifest*. World Wide Web Consortium. https://www.w3.org/TR/appmanifest/

Zod. (2026). *Zod: TypeScript-first schema validation* (Version 4) [Computer software]. https://zod.dev

11. APPENDICES

11.1 GitHub Repository

Repository: **https://github.com/altechemist/civiclink** (public; source code, tests, documentation and CI history).

Submitted version: tag **v1.0.0** (the commit it points to is shown on the repository's Releases and Tags pages).

11.2 Deployed System

URL: **https://civiclink-five.vercel.app** (Vercel + Firebase Spark, decision D2). Demo accounts are listed in Section 8. The local emulator configuration in Section 8 reproduces the same system offline.

11.3 Additional Supporting Material

- **Complete automated test results:** the GitHub Actions run for the v1.0.0 commit (tab "Actions"). It includes the Playwright HTML report and the coverage report as downloadable artefacts.
- **Traceability matrix** (requirement → rule → screen → code → data → test): docs/trackers/traceability-matrix.md.
- **Full test catalogue:** docs/trackers/test-catalog.md; **defect log:** docs/trackers/defect-log.md.
- **Baseline change log** reconciling Phases 1–3 (CR-01 to CR-45): docs/baseline/change-log.md.
- **Architecture decision records:** docs/adr/README.md.
- **Phase 1–3 documents:** docs/phases/; **diagrams:** docs/diagrams/; **all UI screenshots:** docs/evidence/.

Abbreviations

Table 23: Abbreviations

Term	Meaning
AC	Acceptance criteria
ADR	Architecture decision record
AGSA	Auditor-General of South Africa
API	Application programming interface
BR	Business rule
CDN	Content delivery network (Vercel's edge cache)
CI	Continuous integration (GitHub Actions)
CR	Change request (baseline change log)
CSP	Content Security Policy (browser rules for which scripts and connections a page may use)
E.164	International telephone number format (+27…)
E2E	End-to-end test
FCM	Firebase Cloud Messaging
FR / NFR	Functional / non-functional requirement
HSTS	HTTP Strict Transport Security (HTTPS only)
IDP	Integrated development plan

Term	Meaning
MFMA	Municipal Finance Management Act
POPIA	Protection of Personal Information Act 4 of 2013
PWA	Progressive web app (installable, works offline)
SC	Success criterion
SLA	Service-level agreement (target resolution time per department)
SMS	Short message service (text message)
TX	Critical database transaction (TX-1 submit, TX-2 assign, TX-3 complete)
WCAG	Web Content Accessibility Guidelines

Requirements traceability

Every included requirement, with its business rules, screen and the tests that verify it. The full matrix, including components and data, is in docs/trackers/traceability-matrix.md.

Table 24: Requirements traceability (condensed from docs/trackers/traceability-matrix.md)

FR	Business rules	Screen	Verified by	Status
FR-01	BR-16	/register	UT-SCH-05, IT-AUTH-01, IT-AUTH-02	Included
FR-02	BR-15, BR-16	/login, all routes	IT-AUTH-03, RT-USR-05, E2E-04, E2E-06	Included
FR-03	BR-01, BR-02	/resident/report/new	UT-SCH-01..04, UT-SCH-09, IT-RPT-01..06, IT-RPT-10, RT-PHO-01, E2E-05, E2E-07	Included
FR-04	BR-04, BR-05	submit result	UT-REF-01, IT-RPT-07, IT-RPT-08	Included
FR-05	BR-03	duplicate result	UT-DUP-01..06, IT-RPT-09, E2E-02	Included
FR-06	BR-17	/resident, /resident/reports, /resident/reports/:id	RT-RPT-02, RT-RPT-03, E2E-01	Included
FR-07	BR-17	/map	UT-PRV-01, RT-PUB-01, IT-RPT-07	Included
FR-08	BR-20	/official, /official/reports	RT-RPT-04, RT-RPT-06, E2E-01	Included
FR-09	BR-05, BR-12, BR-19, BR-20	/official/reports/:id	IT-TRI-01..05	Included
FR-10	BR-06,	/official/reports/:id	IT-ASG-01..07, E2E-01	Included

FR	Business rules	Screen	Verified by	Status
	BR-07, BR-20			
FR-11	BR-08, BR-09	/worker, /worker/tasks/:id	UT-TSM-01, UT-SCH-06, IT-TSK-01..07, RT-PHO-01, E2E-01	Included
FR-12	BR-06, BR-07	/official/reports/:id	IT-ASG-08..10, IT-TSK-06	Included
FR-13	BR-10	queue, dashboard	UT-SLA-01..02, IT-ESC-01..03	Included
FR-14	BR-11	/resident/reports/:id	UT-RES-01, IT-RES-01..05, E2E-01	Included
FR-15	—	NotificationBell (all shells)	IT-RPT-07, IT-ASG-01, IT-TSK-01, RT-USR-04, NFT-05	Included
FR-16	BR-13	/official/alerts/new	UT-SCH-ALR-01..02, IT-ALR-01..04, E2E-03	Included
FR-17	BR-14	/resident/alerts, /resident	RT-ALR-01..02, IT-ALR-05, E2E-03	Included
FR-18	—	/official/alerts	IT-ALR-05, IT-ALR-06	Included
FR-19	—	/official/analytics	UT-ANL-01..02, IT-ANL-01	Included
FR-20	BR-15, BR-16	/admin/users	UT-SCH-07, IT-ADM-01..06, IT-AUTH-03	Included
FR-21	BR-19	/admin/config	UT-SCH-08, IT-CFG-01..04, IT-RPT-05	Included
FR-22	BR-18	/official/reports/:id (internal notes)	IT-CMT-01, RT-CMT-01	Included
FR-23	—	/admin/logs	IT-ADM-07, RT-CFG-01	Included
FR-24	—	/profile (push toggle)	IT-PSH-01..03; manual live check	Included
FR-26	—	queue "Export CSV"	UT-CSV-01	Included
FR-27	BR-01, BR-02	/resident/report/new, /resident (saved reports)	UT-OFF-01, IT-OFF-01, IT-OFF-02, E2E-12	Included
FR-28	BR-13, BR-24	/official/alerts/new, /official/alerts?tab=scheduled	UT-ALS-01..02, IT-SCH-01..02, RT-ALS-01, E2E-13	Included
FR-29	BR-16	/login, /welcome	IT-AUTH-04, IT-AUTH-05, E2E-08	Included
FR-30	—	/login	E2E-09	Included
FR-31	BR-14, BR-16	/profile	IT-PRF-01, IT-PRF-02, E2E-10	Included
FR-33	BR-23	/map (popup)	IT-AFF-01, IT-AFF-02, E2E-11	Included

FR	Business rules	Screen	Verified by	Status
FR-34	—	all (offline banner), /resident (install)	NFT-10	Included
FR-35	BR-06, BR-07, BR-20	/official/reports (checkboxes)	UT-BLK-01, IT-BLK-01..02, E2E-14	Included
FR-36	BR-17	/transparency (public)	UT-PUB-01, UT-RAT-02, IT-PUB-01..02, E2E-15, NFT-11	Included
FR-37	BR-11	/resident/reports/:id (stars)	UT-RAT-01, IT-RAT-01, E2E-01	Included
FR-38	BR-17	/ (signed out)	IT-PUB-03, E2E-16, NFT-08	Included
FR-39	BR-01, BR-26	/report, /account	UT-GST-01, IT-GST-01..05, RT-GST-01, E2E-16, E2E-17	Included
FR-32	BR-22	/profile	IT-DEL-01, IT-DEL-02, E2E-10	Included

Complete automated test results

Collected from the output of every test suite in the latest full run.

Run 2026-11-06 07:07 UTC on commit `e53abd1`: **185 of 185 passed**.

Table 25: Complete automated test results

Test ID	Scenario	Suite	Result
UT-RSM-01	every allowed transition succeeds for its allowed actors	Unit	Pass
UT-RSM-02	every other (status, event) pair is rejected	Unit	Pass
UT-RSM-03	terminal statuses allow no events and actors are enforced	Unit	Pass
UT-TSM-01	tasks move in order; skipping steps is rejected (BR-08)	Unit	Pass
UT-DUP-01	distance boundary 99 m / 100 m / 101 m	Unit	Pass
UT-DUP-02	time window 71 h / 73 h	Unit	Pass
UT-DUP-03	different category is not a duplicate	Unit	Pass
UT-DUP-04	parent must be open	Unit	Pass
UT-DUP-05	picks the closest candidate, oldest on a tie	Unit	Pass
UT-DUP-06	haversine distance sanity check (Kimberley CBD → Galeshewe ≈ 3.4 km)	Unit	Pass
UT-SLA-01	breach only after slaHours	Unit	Pass
UT-SLA-02	never re-escalates and ignores closed work	Unit	Pass
UT-RES-01	7-day window and auto-close	Unit	Pass

Test ID	Scenario	Suite	Result
UT-REF-01	reference format and padding (BR-04)	Unit	Pass
UT-PRV-01	public location is rounded to ~110 m (BR-17)	Unit	Pass
UT-ANL-01	metrics on a fixture	Unit	Pass
UT-ANL-02	empty input yields nulls, not NaN	Unit	Pass
UT-CSV-01	CSV quoting	Unit	Pass
UT-LOC-01	municipal bounds and nearest suburb	Unit	Pass
UT-ALS-01	publish time must be 10 min to 30 days ahead and before expiry	Unit	Pass
UT-ALS-02	publishAt is optional in the broadcast schema	Unit	Pass
UT-OFF-01	clientId is optional and limited to a safe id format	Unit	Pass
UT-BLK-01	one to 20 reports per batch	Unit	Pass
UT-RAT-01	ratings are whole stars from 1 to 5, with optional short feedback	Unit	Pass
UT-RAT-02	average to one decimal; no ratings gives null; invalid values ignored	Unit	Pass
UT-PUB-01	open reports per suburb exclude closed, rejected and duplicate reports	Unit	Pass
UT-SMS-01	South African numbers convert to E.164	Unit	Pass
UT-SMS-02	SMS text fits one 160-character segment	Unit	Pass
UT-SMS-03	opting in to SMS requires a mobile number	Unit	Pass
UT-GST-01	an upgrading guest gives a name and suburb; transfer codes have a fixed safe format	Unit	Pass
UT-SCH-01	description boundaries 9/10/300/301	Unit	Pass
UT-SCH-02	photo count 0/3/4 and file size and type	Unit	Pass
UT-SCH-03	location inside / outside municipal bounding box	Unit	Pass
UT-SCH-04	rejects unknown suburb and severity with field errors	Unit	Pass
UT-SCH-05	password minimum, optional SA phone, no role field accepted	Unit	Pass
UT-SCH-06	completion requires notes ≥10 and 1–3 proof photos (BR-09)	Unit	Pass
UT-SCH-ALR-01	title/message boundaries and target suburbs (BR-13)	Unit	Pass
UT-SCH-ALR-02	expiry must be future and within 30 days	Unit	Pass
UT-SCH-07	staff users need a department unless admin	Unit	Pass
UT-SCH-08	department SLA hours 1–720 whole numbers	Unit	Pass
UT-SCH-09	real type is detected from magic bytes	Unit	Pass

Test ID	Scenario	Suite	Result
RT-USR-01	a user reads their own profile and may only stamp lastLoginAt	Security rules	Pass
RT-USR-02	users cannot change their own role, department or active flag (BR-16)	Security rules	Pass
RT-USR-03	residents cannot read other users; staff read the staff directory only	Security rules	Pass
RT-USR-04	notifications are private; only readAt can be set	Security rules	Pass
RT-USR-05	deactivated or anonymous users read nothing	Security rules	Pass
RT-RPT-01	no client can create or modify reports or history (functions only)	Security rules	Pass
RT-RPT-02	the reporter and duplicate followers read the report and its history	Security rules	Pass
RT-RPT-03	residents cannot read others' reports or list all reports (BR-17)	Security rules	Pass
RT-RPT-04	officials read only their department (BR-20)	Security rules	Pass
RT-RPT-05	workers read only reports assigned to them	Security rules	Pass
RT-RPT-06	admins read every report	Security rules	Pass
RT-CMT-01	internal comments are hidden from residents (BR-18)	Security rules	Pass
RT-PUB-01	the public projection is readable by residents, officials and admins only	Security rules	Pass
RT-TSK-01	workers read own tasks; officials read their department; others denied	Security rules	Pass
RT-ALR-01	signed-in users read alerts; nobody writes them directly	Security rules	Pass
RT-ALR-02	residents read only their own delivery record; officials read all	Security rules	Pass
RT-CFG-01	configuration is readable, never client-writable; counters and logs locked	Security rules	Pass
RT-ALS-01	scheduled alerts are visible to staff only until published; nobody writes them directly (FR-28)	Security rules	Pass
RT-GST-01	guest transfer codes and rate-limit counters are server-only (FR-39)	Security rules	Pass
RT-SMS-01	SMS records (phone numbers) are readable by administrators only (FR-25, NFR-11)	Security rules	Pass
RT-PHO-01	report photos and proof are visible to the reporter, followers and staff with access (BR-17)	Security rules	Pass
RT-PHO-02	other users cannot read photos, and nobody writes	Security rules	Pass

Test ID	Scenario	Suite	Result
	them directly		
IT-AUTH-04	completeProfile creates a RESIDENT profile and claims once	Integration	Pass
IT-AUTH-05	existing accounts and anonymous callers cannot use completeProfile	Integration	Pass
IT-PRF-01	residents change suburb and phone; claims follow; invalid phone refused	Integration	Pass
IT-PRF-02	staff may change name and phone but have no home suburb	Integration	Pass
IT-DEL-01	deletion anonymises reports and removes photos, profile, receipts and the Auth account	Integration	Pass
IT-DEL-02	staff cannot self-delete; anonymised reports still progress without notifying anyone	Integration	Pass
IT-RPT-11	submissions beyond the hourly limit are refused	Integration	Pass
IT-CFG-05	settings are admin-only and bounded	Integration	Pass
IT-AUTH-01	self-registration creates a RESIDENT with claims and a profile	Integration	Pass
IT-AUTH-02	duplicate email and weak password are refused	Integration	Pass
IT-AUTH-03	a deactivated account is refused by every command	Integration	Pass
IT-ALR-01	broadcast creates one recipient per active resident in the target suburbs	Integration	Pass
IT-ALR-02	ALL targets every active resident	Integration	Pass
IT-ALR-03	validation: long title, empty targets, past or far expiry	Integration	Pass
IT-ALR-04	residents and field workers cannot broadcast	Integration	Pass
IT-ALR-05	mark-as-read counts once per resident	Integration	Pass
IT-ALR-06	deactivation hides the alert	Integration	Pass
IT-ADM-01	admin creates a staff account with claims	Integration	Pass
IT-ADM-02	staff accounts need an active department	Integration	Pass
IT-ADM-03	role change updates the profile and claims	Integration	Pass
IT-ADM-04	an admin cannot demote or deactivate themselves (BR-15)	Integration	Pass
IT-ADM-05	non-admins cannot manage users	Integration	Pass
IT-ADM-06	reactivation restores access	Integration	Pass
IT-ADM-07	admin actions are written to the system log (FR-23)	Integration	Pass
IT-CFG-01	SLA hours must be 1–720	Integration	Pass
IT-CFG-02	names are unique (case-insensitive)	Integration	Pass
IT-CFG-03	categories link to an existing department and can be	Integration	Pass

Test ID	Scenario	Suite	Result
	deactivated		
IT-CFG-04	non-admins cannot change configuration	Integration	Pass
IT-ANL-01	metrics reflect live data and are restricted to officials and admins	Integration	Pass
IT-OFF-01	a report re-sent with the same clientId is stored once and returns the same reference	Integration	Pass
IT-OFF-02	another resident cannot reuse a clientId to read someone else's report; malformed ids are refused	Integration	Pass
IT-SCH-01	a scheduled alert is invisible until due, then published once with the same id	Integration	Pass
IT-SCH-02	invalid times are refused; a cancelled alert is never published	Integration	Pass
IT-BLK-01	assigns every eligible report and reports the rest without blocking them	Integration	Pass
IT-BLK-02	other departments, residents and oversized batches are refused	Integration	Pass
IT-RAT-01	the reporter rates the fix when confirming; out-of-range ratings are refused	Integration	Pass
IT-PUB-01	anyone can GET anonymous figures, with CDN cache headers	Integration	Pass
IT-PUB-02	the figures are only readable, not writable	Integration	Pass
IT-SMS-01	emergency alerts are queued by SMS for opted-in residents only; other types are not	Integration	Pass
IT-SMS-02	a gateway failure is recorded per message and logged, and does not fail the alert	Integration	Pass
IT-SMS-03	removing the number ends the opt-in; deleting the account deletes the SMS records	Integration	Pass
IT-SMS-04	an administrator can send a test SMS (simulated without credentials); others cannot	Integration	Pass
IT-SMS-05	with Twilio credentials, messages go to Twilio's REST API in the documented form (trial: a predefined template); errors are recorded	Integration	Pass
IT-GST-01	an anonymous session becomes a guest Resident; starting again changes nothing	Integration	Pass
IT-GST-02	a guest can report and follow it, within a lower hourly limit; account-only actions are refused	Integration	Pass
IT-GST-03	a guest creates an account in the same session and keeps their reports	Integration	Pass

Test ID	Scenario	Suite	Result
IT-GST-04	a guest moves their reports into an existing account with a one-time code	Integration	Pass
IT-GST-05	guest sessions are limited per network address	Integration	Pass
IT-PUB-03	anyone can GET open issues (approximate) and active alerts, cached for a minute	Integration	Pass
IT-HTTP-01	a resident submits a report through the HTTP API	Integration	Pass
IT-HTTP-02	authentication, roles and validation behave exactly as with Cloud Functions	Integration	Pass
IT-HTTP-03	unknown commands and wrong methods are refused	Integration	Pass
IT-HTTP-04	the cron endpoint requires the secret	Integration	Pass
IT-PSH-01	devices register and unregister; tokens are stored server-side only; anonymous refused	Integration	Pass
IT-PSH-02	a committed status change pushes once to the reporter; a refused command pushes nothing	Integration	Pass
IT-PSH-03	suburb alerts are pushed to targeted residents and dead tokens are pruned	Integration	Pass
IT-RPT-01	valid submission creates a SUBMITTED report with a reference	Integration	Pass
IT-RPT-02	invalid fields are rejected with field errors and nothing is written	Integration	Pass
IT-RPT-03	location outside the municipality is rejected (BR-02)	Integration	Pass
IT-RPT-04	photos must really be images of the declared type and within the size limit	Integration	Pass
IT-RPT-05	inactive category is refused (BR-19)	Integration	Pass
IT-RPT-06	only residents may submit (BR-01)	Integration	Pass
IT-RPT-07	default department from category, one history entry, officials notified	Integration	Pass
IT-RPT-08	concurrent submissions get unique sequential references (BR-04, NFR-10)	Integration	Pass
IT-RPT-09	a nearby open report of the same category is linked as a duplicate (BR-03)	Integration	Pass
IT-RPT-10	optional fields sent as undefined/null by the web client are accepted (DEF-02)	Integration	Pass
IT-TRI-01	triage moves SUBMITTED → UNDER_REVIEW and records history	Integration	Pass
IT-TRI-02	triage from a non-SUBMITTED status is refused	Integration	Pass
IT-TRI-03	reject needs a reason ≥ 10 characters (BR-12)	Integration	Pass

Test ID	Scenario	Suite	Result
IT-TRI-04	officials cannot act on another department's report (BR-20)	Integration	Pass
IT-TRI-05	rerouting to an inactive department is refused; active reroute works	Integration	Pass
IT-ASG-01	assignment creates a task, moves the report to IN_PROGRESS and notifies	Integration	Pass
IT-ASG-02	a worker from another department is refused (BR-07)	Integration	Pass
IT-ASG-03	an inactive worker is refused (BR-07)	Integration	Pass
IT-ASG-04	a report with an active task cannot be assigned again (BR-06)	Integration	Pass
IT-ASG-05	a SUBMITTED (untriaged) report cannot be assigned	Integration	Pass
IT-ASG-06	residents and workers cannot assign	Integration	Pass
IT-ASG-07	two concurrent assignments: exactly one succeeds (NFR-10)	Integration	Pass
IT-ASG-08	reassignment ends the old task and creates a new one	Integration	Pass
IT-ASG-09	reassigning to the same worker is refused	Integration	Pass
IT-ASG-10	reassigning without an active task is refused	Integration	Pass
IT-TSK-01	accept → start → complete resolves the report and notifies the resident	Integration	Pass
IT-TSK-02	another worker cannot act on the task (BR-08)	Integration	Pass
IT-TSK-03	steps cannot be skipped (BR-08)	Integration	Pass
IT-TSK-04	completion without notes or proof is refused (BR-09)	Integration	Pass
IT-TSK-05	proof photos must match their declared type	Integration	Pass
IT-TSK-06	a reassigned (stale) task cannot be completed	Integration	Pass
IT-TSK-07	officials and residents cannot complete tasks	Integration	Pass
IT-ESC-01	the schedule escalates reports past their SLA once, and is idempotent	Integration	Pass
IT-ESC-02	manual escalation requires a reason; escalated reports can be assigned	Integration	Pass
IT-ESC-03	only admins can run maintenance	Integration	Pass
IT-RES-01	the reporter can confirm a resolved report (→ CLOSED)	Integration	Pass
IT-RES-02	the reporter can reopen with a reason (→ UNDER_REVIEW); officials notified	Integration	Pass
IT-RES-03	after 7 days confirm/reopen are refused and the	Integration	Pass

Test ID	Scenario	Suite	Result
	schedule closes the report		
IT-RES-04	another resident cannot confirm or reopen	Integration	Pass
IT-RES-05	confirming an unresolved report is refused	Integration	Pass
IT-AFF-01	a resident confirms an open issue: count, follower and timeline update once	Integration	Pass
IT-AFF-02	own reports, closed reports and staff are refused	Integration	Pass
IT-CMT-01	staff with access can comment; residents and other departments cannot	Integration	Pass
E2E-08	first Google sign-in completes a resident profile (FR-29, BR-16)	E2E (chromium)	Pass
E2E-09	password reset gives the same answer whether or not the account exists (FR-30)	E2E (chromium)	Pass
E2E-10	profile change and POPIA self-deletion (FR-31, FR-32)	E2E (chromium)	Pass
E2E-01	full lifecycle: resident → official → field worker → resident (SC-01, NFT-05)	E2E (chromium)	Pass
E2E-02	a second resident reporting the same issue is linked as a duplicate (FR-05)	E2E (chromium)	Pass
E2E-03	suburb-targeted alert reaches Galeshewe but not Roodepan (FR-16, FR-17, BR-14)	E2E (chromium)	Pass
E2E-04	each role is denied other roles' areas (NFR-01)	E2E (chromium)	Pass
E2E-05	invalid report input is rejected with field messages (BR-02, NFR-03)	E2E (chromium)	Pass
E2E-06	wrong password shows a generic error (FR-02)	E2E (chromium)	Pass
E2E-07	GPS outside Sol Plaatje is explained, not used; tapping the map sets the pin and suburb (FR-03, BR-02)	E2E (chromium)	Pass
E2E-11	"Me too" from the community map raises the affected count (FR-33, BR-23)	E2E (chromium)	Pass
NFT-08	accessibility of public home, transparency, login, report form, my reports and queue	E2E (chromium)	Pass
NFT-06	responsiveness: screens interactive < 3 s, p95 command < 2 s	E2E (chromium)	Pass
E2E-12	a report made offline is saved on the phone and sent automatically on reconnect (FR-27)	E2E (chromium)	Pass
E2E-13	an official schedules an alert for later and cancels it (FR-28)	E2E (chromium)	Pass
E2E-14	an official assigns several reports to one field	E2E (chromium)	Pass

Test ID	Scenario	Suite	Result
	worker at once (FR-35)		
E2E-15	the public transparency page works without signing in (FR-36)	E2E (chromium)	Pass
E2E-16	a visitor sees what is happening, reports as a guest, then creates an account and keeps the report (FR-38, FR-39)	E2E (chromium)	Pass
E2E-17	a guest who already has an account signs in and their report moves to it (FR-39)	E2E (chromium)	Pass
NFT-07	mobile layouts fit a 360 px screen	E2E (mobile-360)	Pass
NFT-10	the production build is installable and opens offline (FR-34)	PWA ()	Pass
NFT-11	CDN caching: immutable assets, cached public figures, uncached app shell (NFR-13)	Deployed site (chromium)	Pass
NFT-12	security headers: CSP, HSTS, no framing, no sniffing (NFR-14)	Deployed site (chromium)	Pass